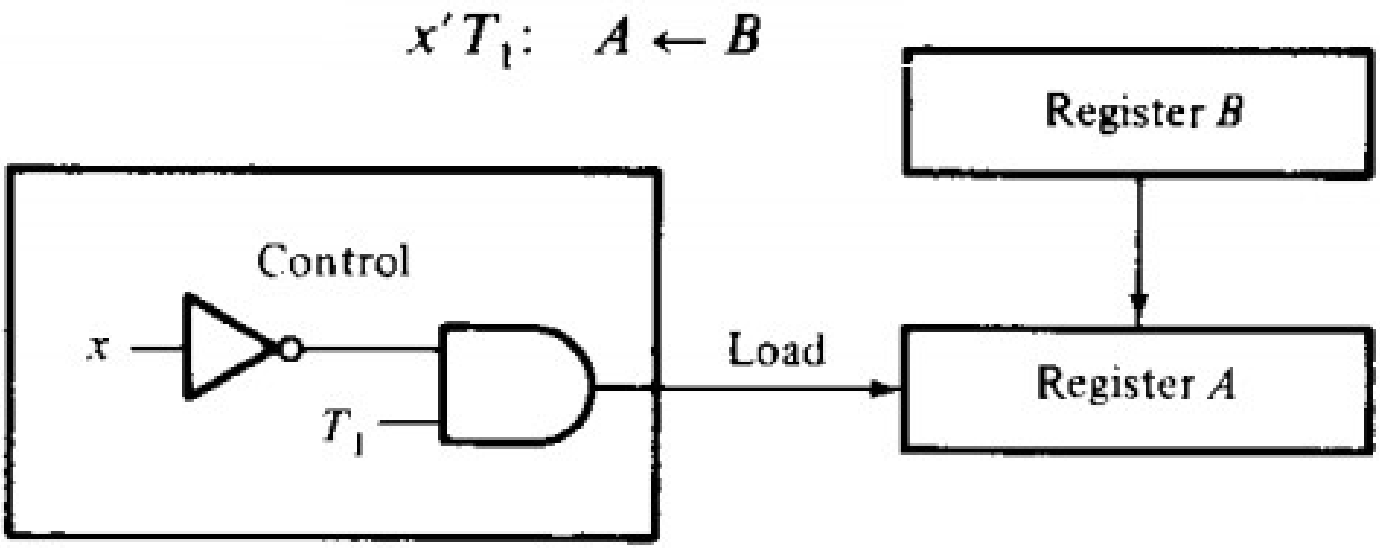
COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Unit IV – Register Transfer Logic

- The following 4 components form the basis of the Register-Transfer Logic Method for describing digital systems:
  1. The set of registers in the system and their functions.
  2. The binary-coded information stored in the registers.
  3. The operations performed on the information stored in the registers.
  4. The control functions that initiate the sequence of operations.



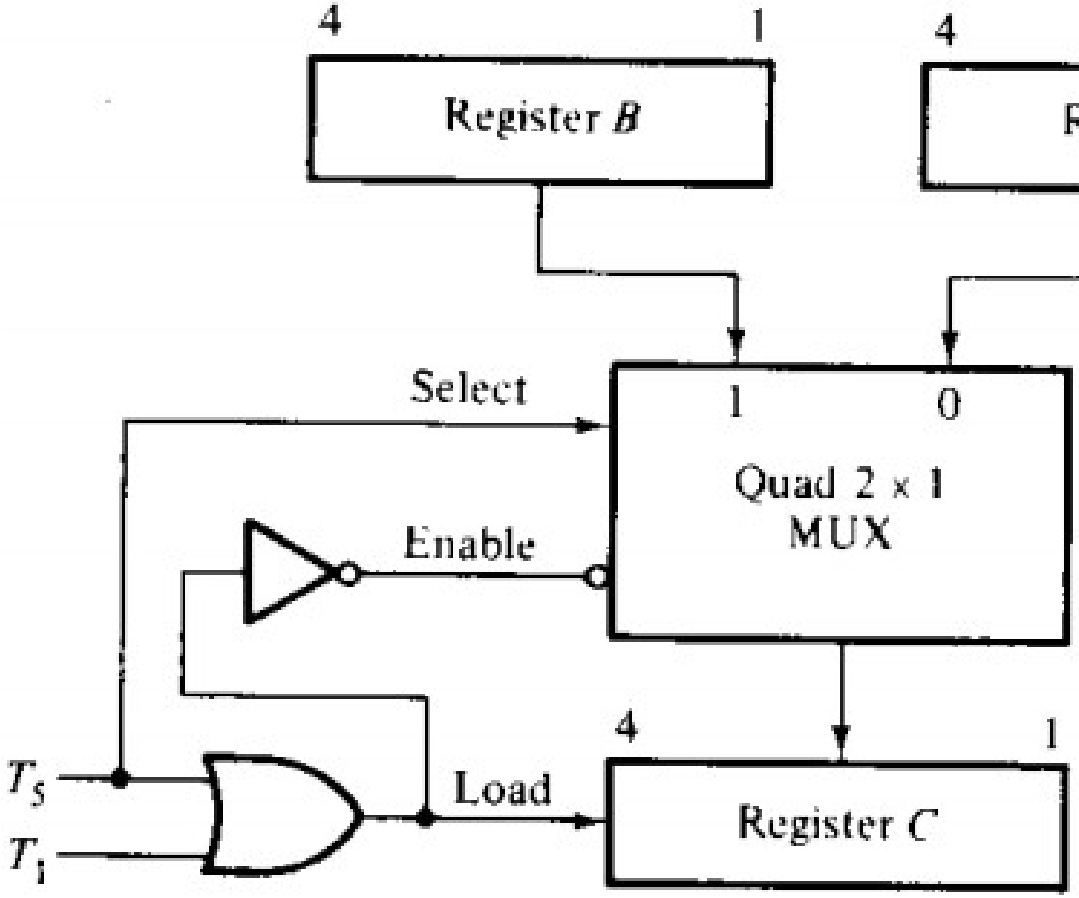
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Basic Symbols for Register Transfer Logic

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	A, MBR, $R2$
Subscript	Denotes a bit of a register	A_2, B_6
Parentheses ( )	Denotes a portion of a register	$PC(H)$, $MBR(OP)$
Arrow $\leftarrow$	Denotes transfer of information	$A \leftarrow B$
Colon :	Terminates a control function	$x'T_0$:
Comma ,	Separates two microoperations	$A \leftarrow B$, $B \leftarrow A$
Square brackets [ ]	Specifies an address for memory transfer	$MBR \leftarrow M[MAR]$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Multiplexer for Transferring Information

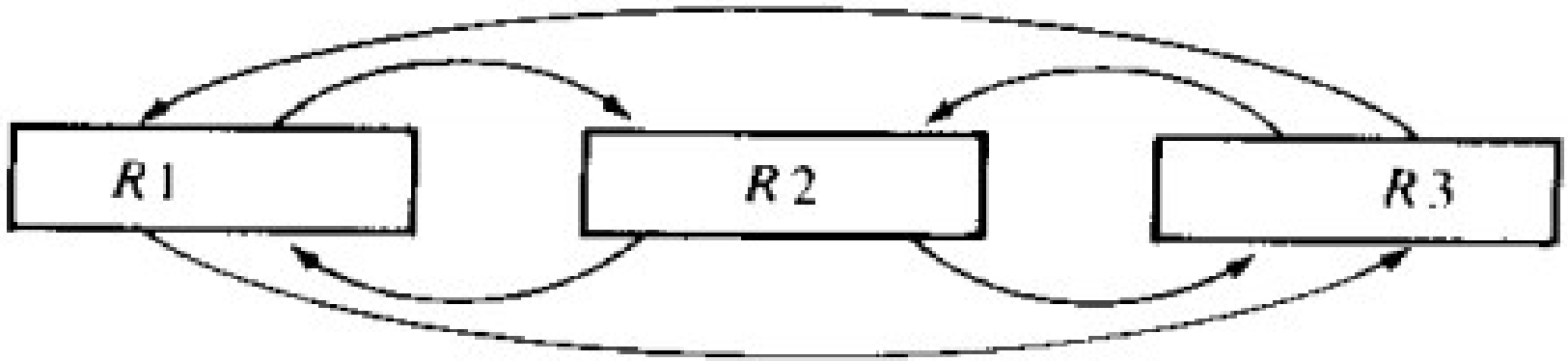


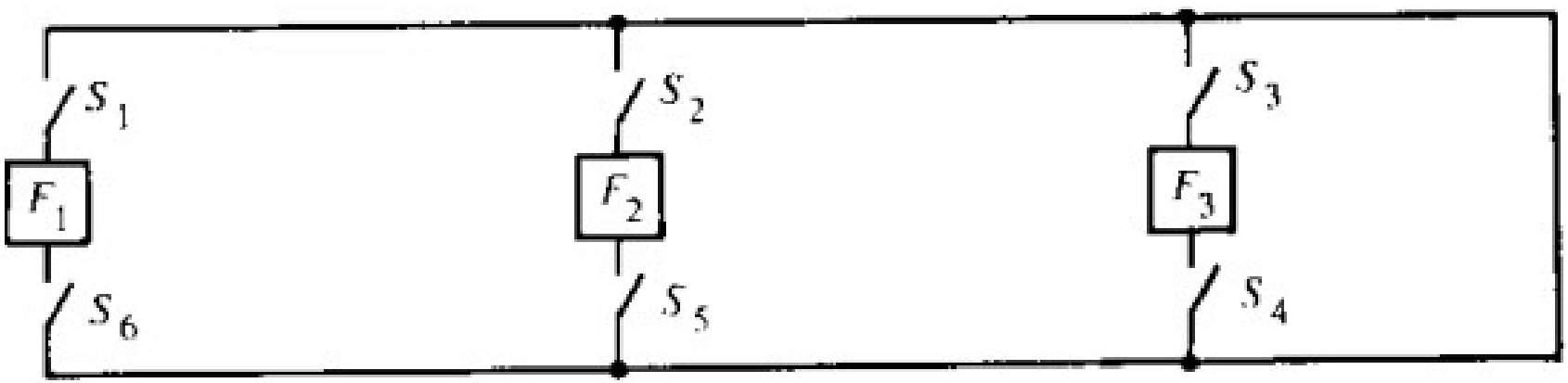
T_1: $C \leftarrow A$

T_5: $C \leftarrow B$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

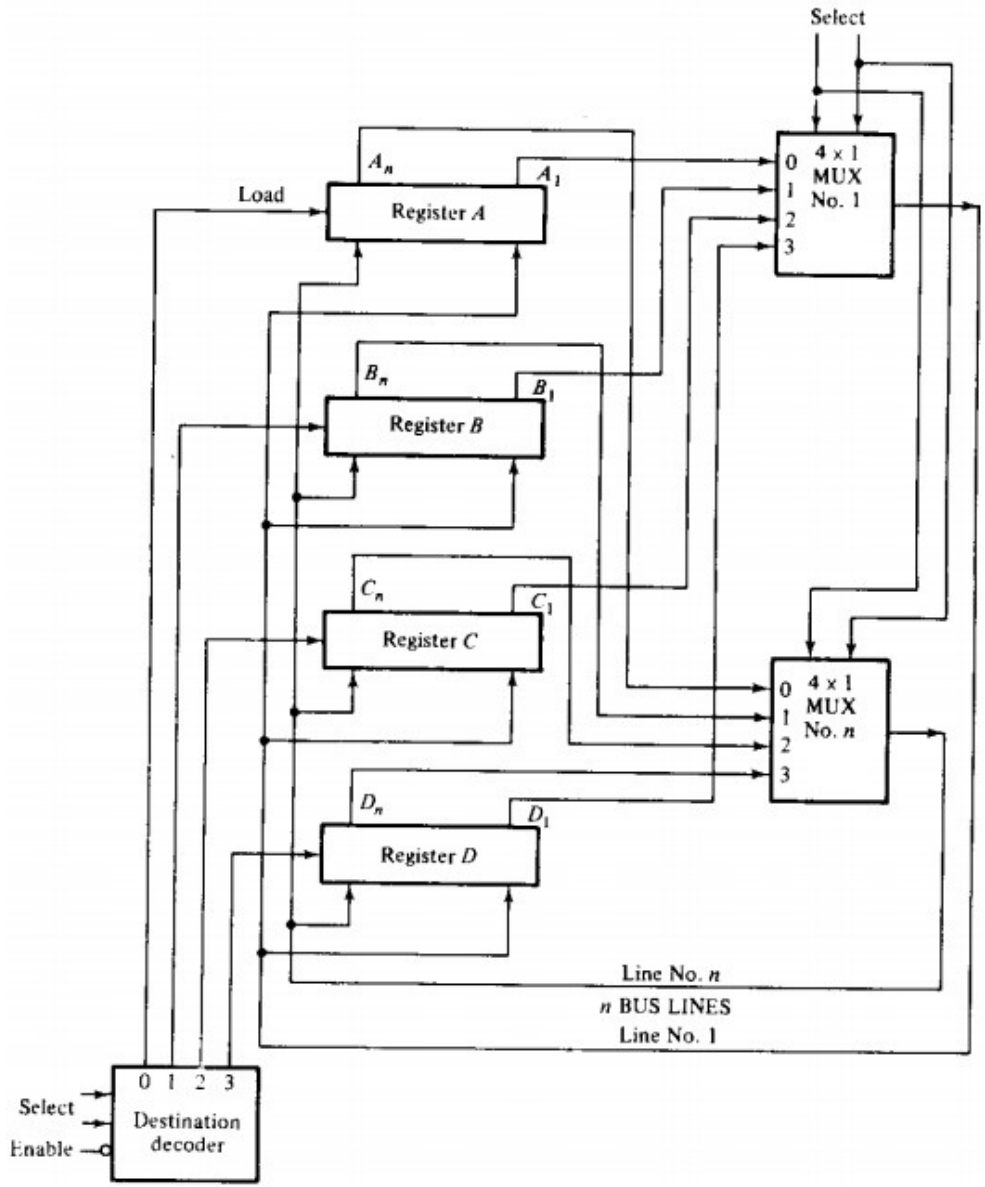
Bus Transfer





Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Bus System for 4 Registers

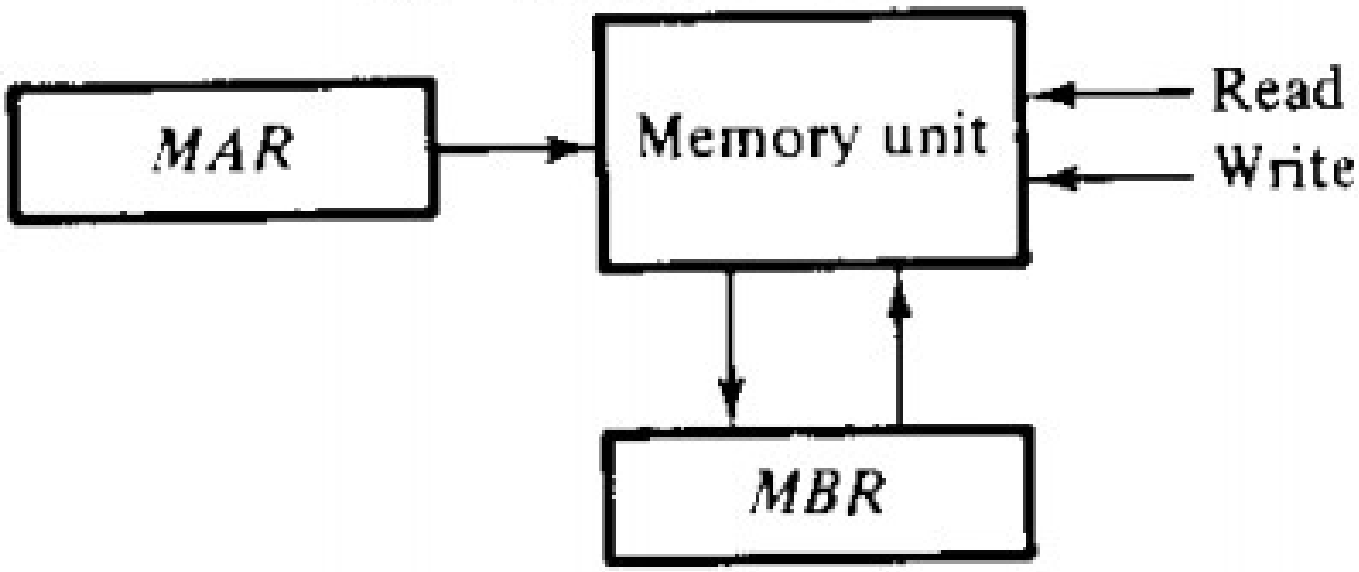


Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Memory Transfer

$$W: \quad M \leftarrow MBR$$

$$R: \quad MBR \leftarrow M$$

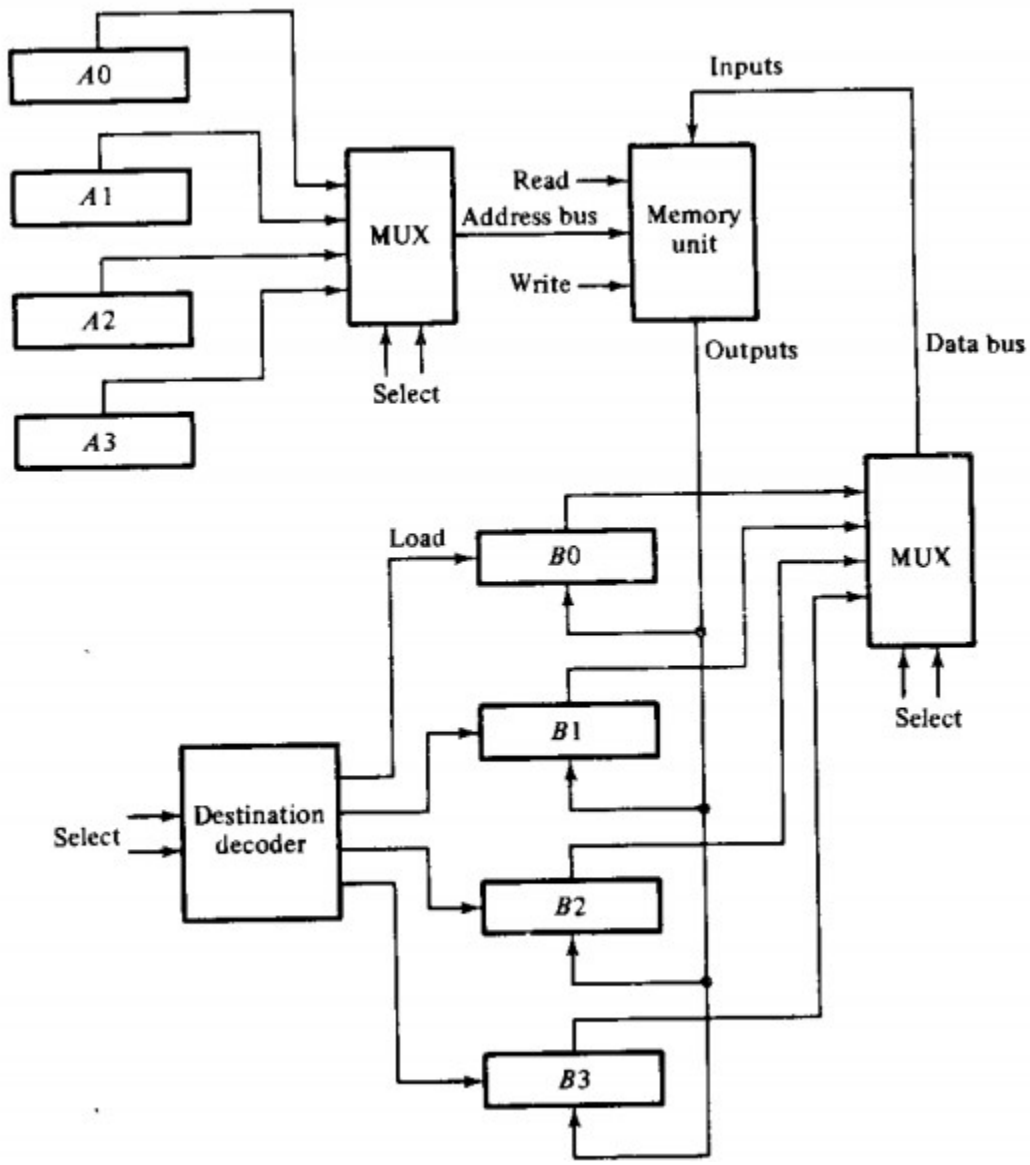


Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Memory Unit with Multiple Registers



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Arithmetic Microoperations

Symbolic designation	Description
$F \leftarrow A + B$	Contents of A plus B transferred to F
$F \leftarrow A - B$	Contents of A minus B transferred to F
$B \leftarrow \overline{B}$	Complement register B (1's complement)
$B \leftarrow \overline{B} + 1$	Form the 2's complement of the contents of register B
$F \leftarrow A + \overline{B} + 1$	A plus the 2's complement of B transferred to F
$A \leftarrow A + 1$	Increment the contents of A by 1 (count up)
$A \leftarrow A - 1$	Decrement the contents of A by 1 (count down)

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Examples of Arithmetic Operations

+ 6	0 000110		− 6	1 111010
		+		+
+ 9	0 001001		+ 9	0 001001
+ 15	0 001111		+ 3	0 000011
+ 6	0 000110		− 9	1 110111
		+		+
− 9	1 110111		− 9	1 110111
− 3	1 111101		− 18	1 101110

$$(\pm A) - (-B) = (\pm A) + (+B)$$

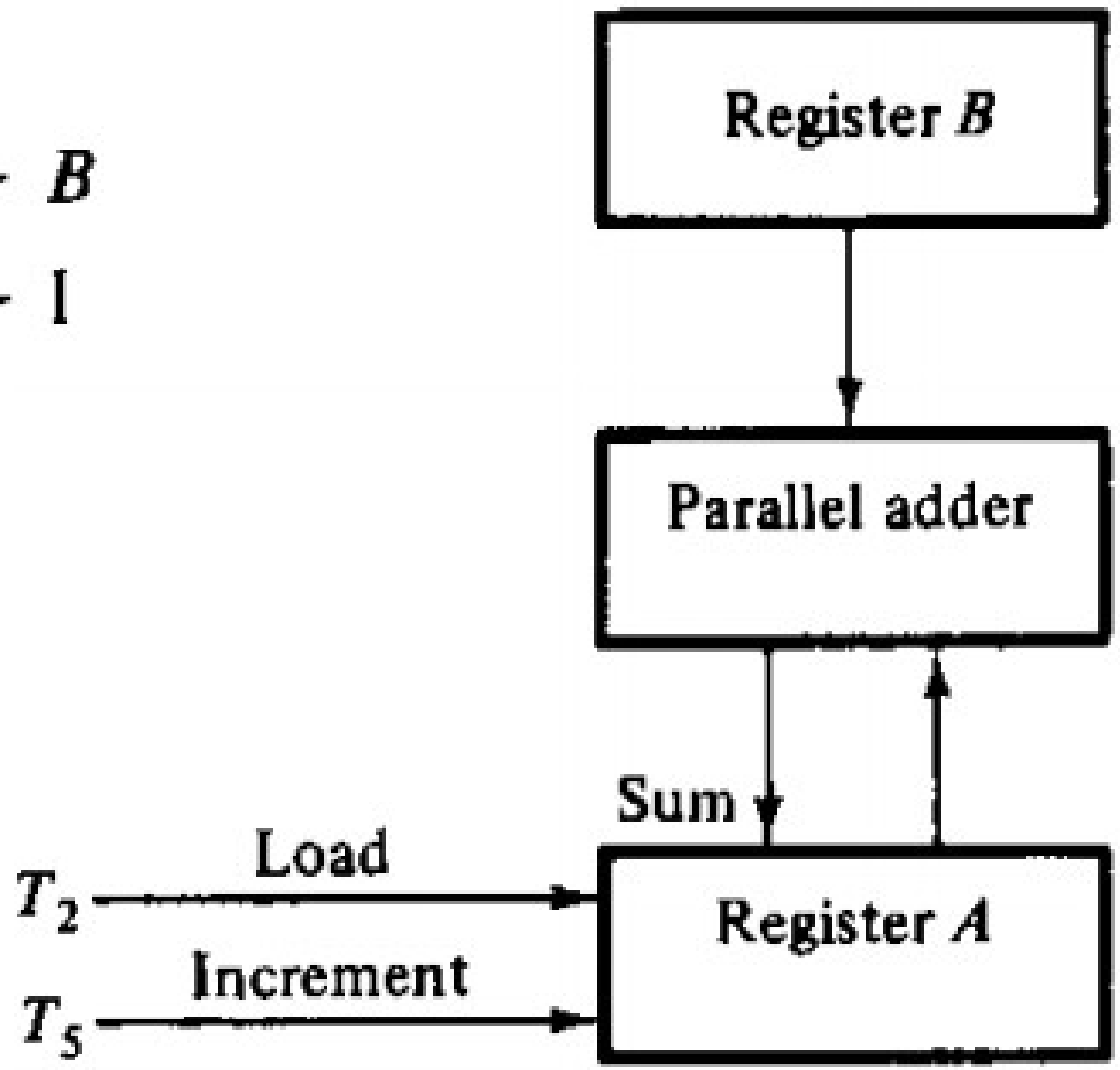
$$(\pm A) - (+B) = (\pm A) + (-B)$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Add and Increment Microoperations

T_2: $A \leftarrow A + B$

T_5: $A \leftarrow A + 1$



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

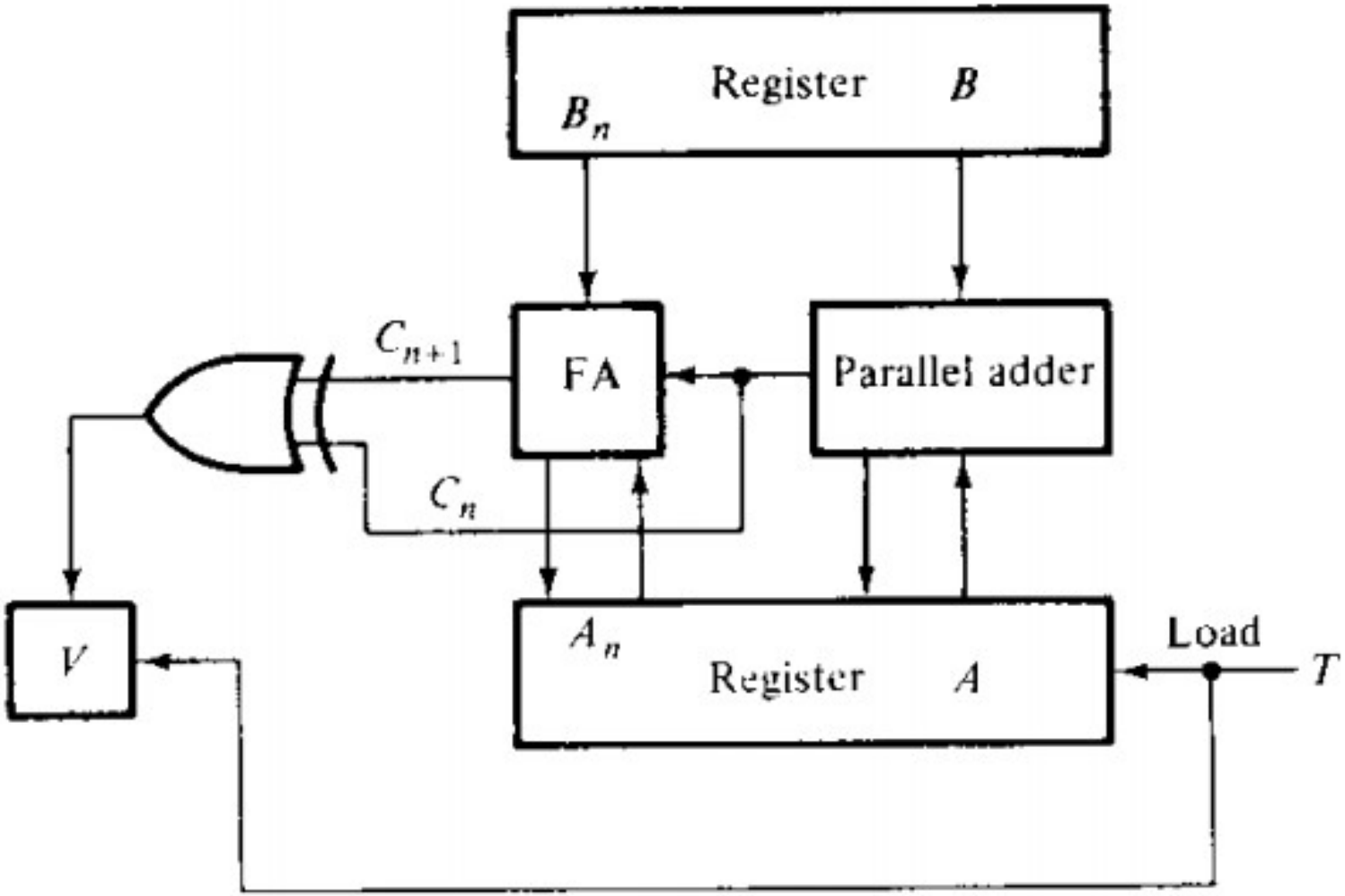
Overflow

carries:	0 1		carries:	1 0
+35	0 100011		−35	1 011101
+40	0 101000		−40	1 011000
+75	1 001011		−75	0 110101

$$T: \quad A \leftarrow A + B, \quad V \leftarrow C_n \oplus C_{n+1}$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Addition of Signed 2's Compliment Numbers



Reference: M. Morris Mano, "Digital Logic and Computer Design", PHI

Logic and Shift Microoperations

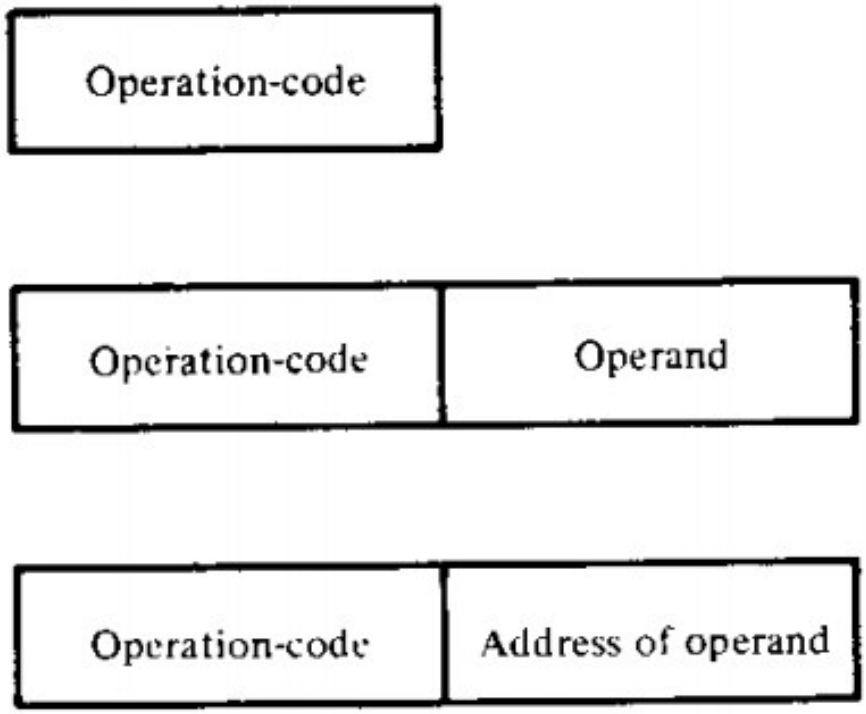
Symbolic designation	Description
$A \leftarrow \overline{A}$	Complement all bits of register A
$F \leftarrow A \vee B$	Logic OR microoperation
$F \leftarrow A \wedge B$	Logic AND microoperation
$F \leftarrow A \oplus B$	Logic exclusive-OR microoperation
$A \leftarrow \text{shl } A$	Shift-left register A
$A \leftarrow \text{shr } A$	Shift-right register A

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Instruction Code Formats



(a) Implied $A \leftarrow R$

(b) Immediate operand $A \leftarrow \text{operand}$

(c) Direct address $A \leftarrow M[\text{address}]$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Example of Instruction Code Formats

Address	Memory		Operation
25	00000001	op-code = 1	$A \leftarrow R$
35	00000010	op-code = 2	$A \leftarrow$ Operand
36	00101100	operand = 44	
45	00000011	op-code = 3	$A \leftarrow M[\text{Address}]$
46	01000110	address = 70	
70	00011100	operand = 28	

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Macrooperation versus Microoperation

- A microoperation runs in one clock cycle
- A statement that may take a sequence of microoperations for its implementation is called a macrooperation.

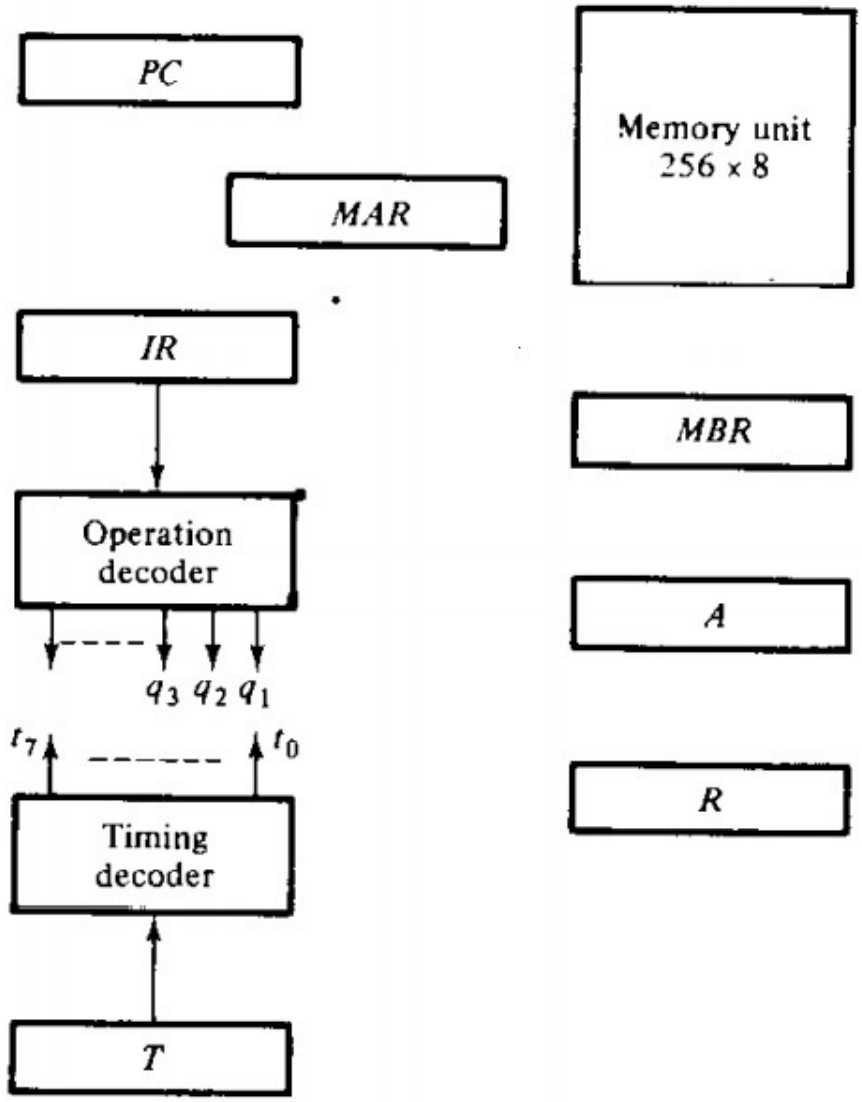
$$A \leftarrow \text{operand}$$

This statement is a macrooperation because it specifies a computer instruction. To execute the instruction, the control unit must issue control functions for the following sequence of microoperations:

1. Read the operation code from address 35.
2. Transfer the operation code to a control register.
3. The control decodes the operation code and recognizes it as an immediate operand instruction, so it reads the operand from address 36.
4. The operand read from memory is transferred into register A.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Building Blocks of a Simple Computer



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

List of Registers for a Simple Computer

Symbol	Number of bits	Name of register	Function
MAR	8	Memory address register	Holds address for memory
MBR	8	Memory buffer register	Holds contents of memory word
A	8	*A* register	Processor register
R	8	*R* register	Processor register
PC	8	Program counter	Holds address of instruction
IR	8	Instruction register	Holds current operation code
T	3	Timing counter	Sequence generator

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

List of Instructions for a Simple Computer

Operation code	Mnemonic		Description	Function
00000001	MOV	R	Move R to A	$A \leftarrow R$
00000010	LDI	OPRD	Load OPRD into A	$A \leftarrow$ OPRD
00000011	LDA	ADRS	Load operand specified by ADRS into A	$A \leftarrow M[\text{ADRS}]$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Instruction Fetch Cycle

macrooperation statement:

$$IR \leftarrow M[PC], \quad PC \leftarrow PC + 1$$

t_0:	$MAR \leftarrow PC$	transfer op-code address
t_1:	$MBR \leftarrow M, PC \leftarrow PC + 1$	read op-code, increment PC
t_2:	$IR \leftarrow MBR$	transfer op-code to IR

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Execution of Instructions

MOV R

q_1t_3: $A \leftarrow R, T \leftarrow 0$

LDI OPRD

q_2t_3:	$MAR \leftarrow PC$	transfer operand address
q_2t_4:	$MBR \leftarrow M, PC \leftarrow PC + 1$	read operand, increment PC
q_2t_5:	$A \leftarrow MBR, T \leftarrow 0$	transfer operand, go to fetch cycle

LDA ADRS

q_3t_3:	$MAR \leftarrow PC$	transfer next instruction address
q_3t_4:	$MBR \leftarrow M, PC \leftarrow PC + 1$	read ADRS, increment PC
q_3t_5:	$MAR \leftarrow MBR$	transfer operand address
q_3t_6:	$MBR \leftarrow M$	read operand
q_3t_7:	$A \leftarrow MBR, T \leftarrow 0$	transfer operand to A, go to fetch cycle

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Register Transfer Statements for a Simple Computer

FETCH	t_0:	$MAR \leftarrow PC$
	t_1:	$MBR \leftarrow M,\ PC \leftarrow PC + 1$
	t_2:	$IR \leftarrow MBR$
MOV	$q_1 t_3$:	$A \leftarrow R,\ T \leftarrow 0$
LDI	$q_2 t_3$:	$MAR \leftarrow PC$
	$q_2 t_4$:	$MBR \leftarrow M,\ PC \leftarrow PC + 1$
	$q_2 t_5$:	$A \leftarrow MBR,\ T \leftarrow 0$
LDA	$q_3 t_3$:	$MAR \leftarrow PC$
	$q_3 t_4$:	$MBR \leftarrow M,\ PC \leftarrow PC + 1$
	$q_3 t_5$:	$MAR \leftarrow MBR$
	$q_3 t_6$:	$MBR \leftarrow M$
	$q_3 t_7$:	$A \leftarrow MBR,\ T \leftarrow 0$

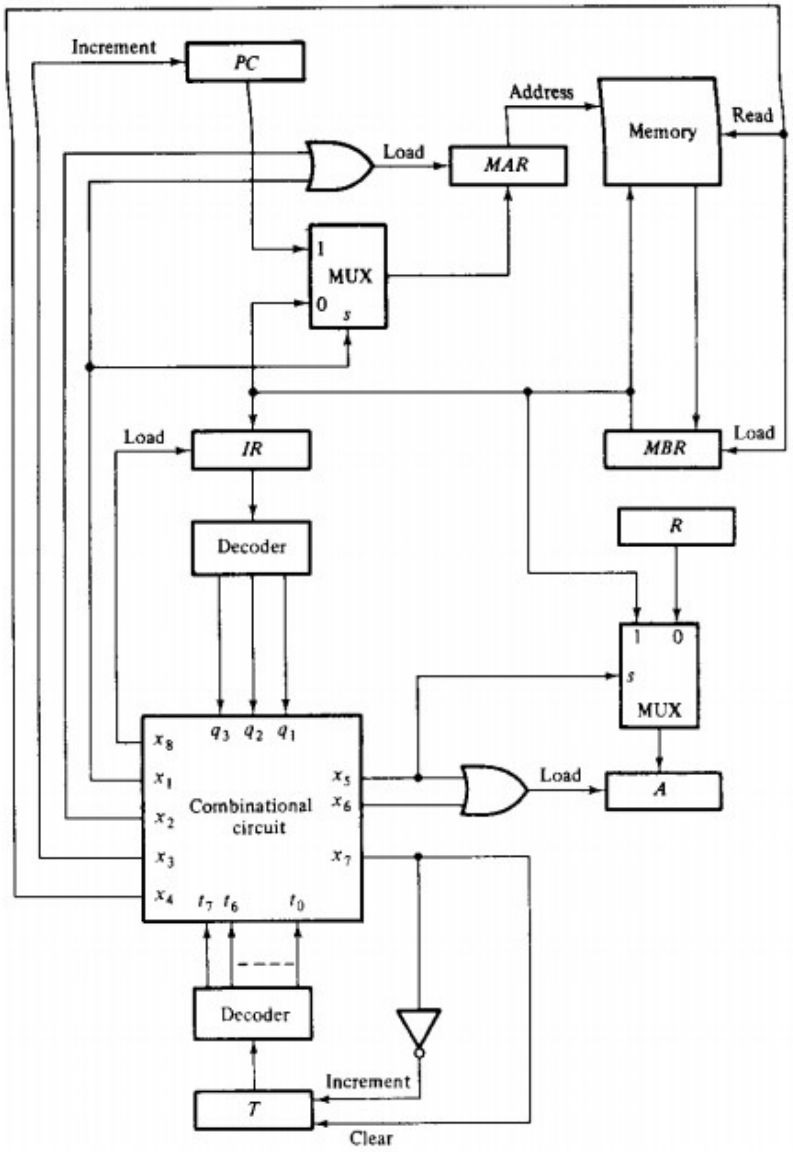
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Hardware Specifications for a Simple Computer

$x_1 = t_0 + q_2t_3 + q_3t_3$:	$MAR \leftarrow PC$
$x_2 = q_3t_5$:	$MAR \leftarrow MBR$
$x_3 = t_1 + q_2t_4 + q_3t_4$:	$PC \leftarrow PC + 1$
$x_4 = x_3 + q_3t_6$:	$MBR \leftarrow M$
$x_5 = q_2t_5 + q_3t_7$:	$A \leftarrow MBR$
$x_6 = q_1t_5$:	$A \leftarrow R$
$x_7 = x_5 + x_6$:	$T \leftarrow 0$
$x_8 = t_2$:	$IR \leftarrow MBR$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Design of a Simple Computer

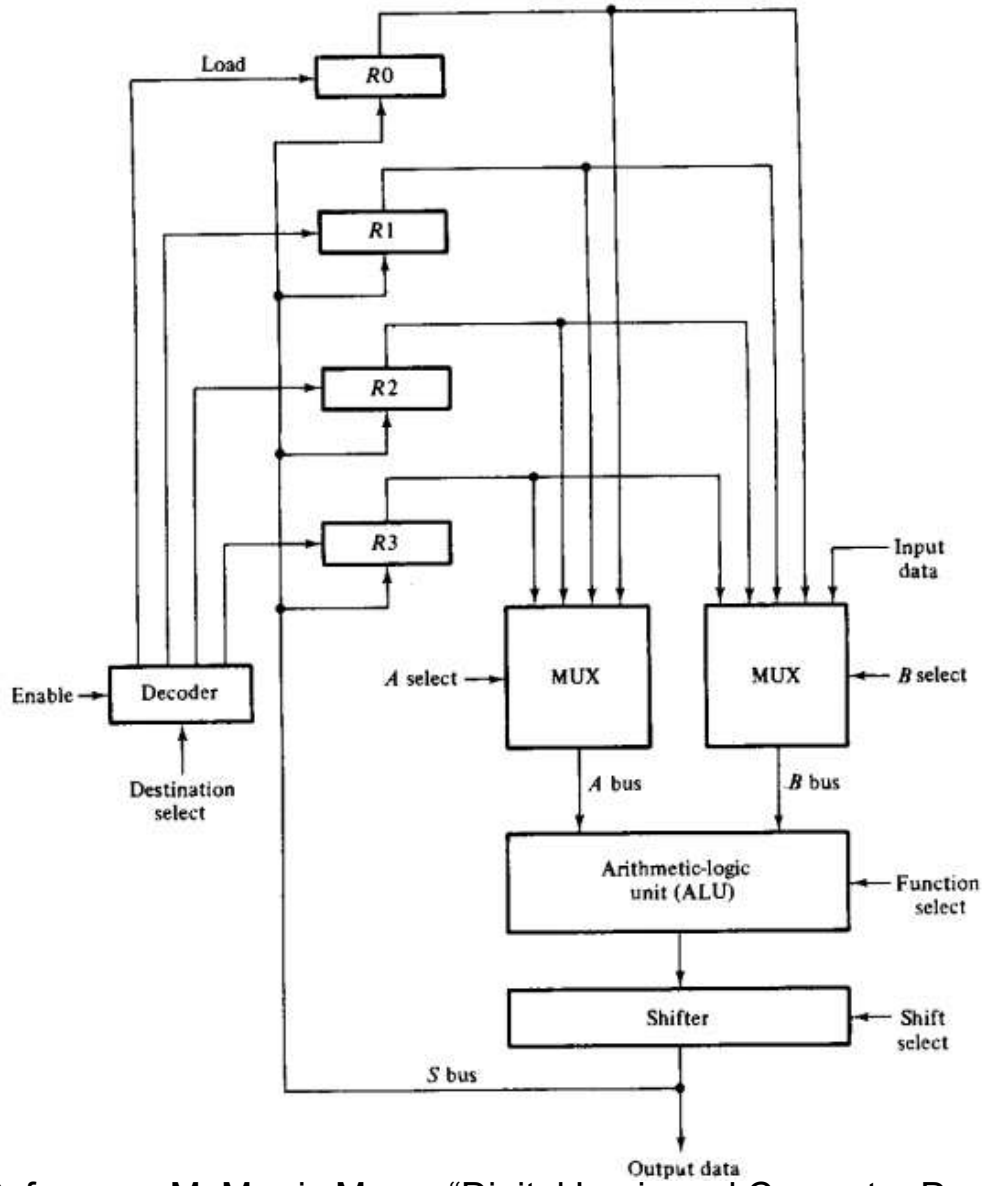


Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

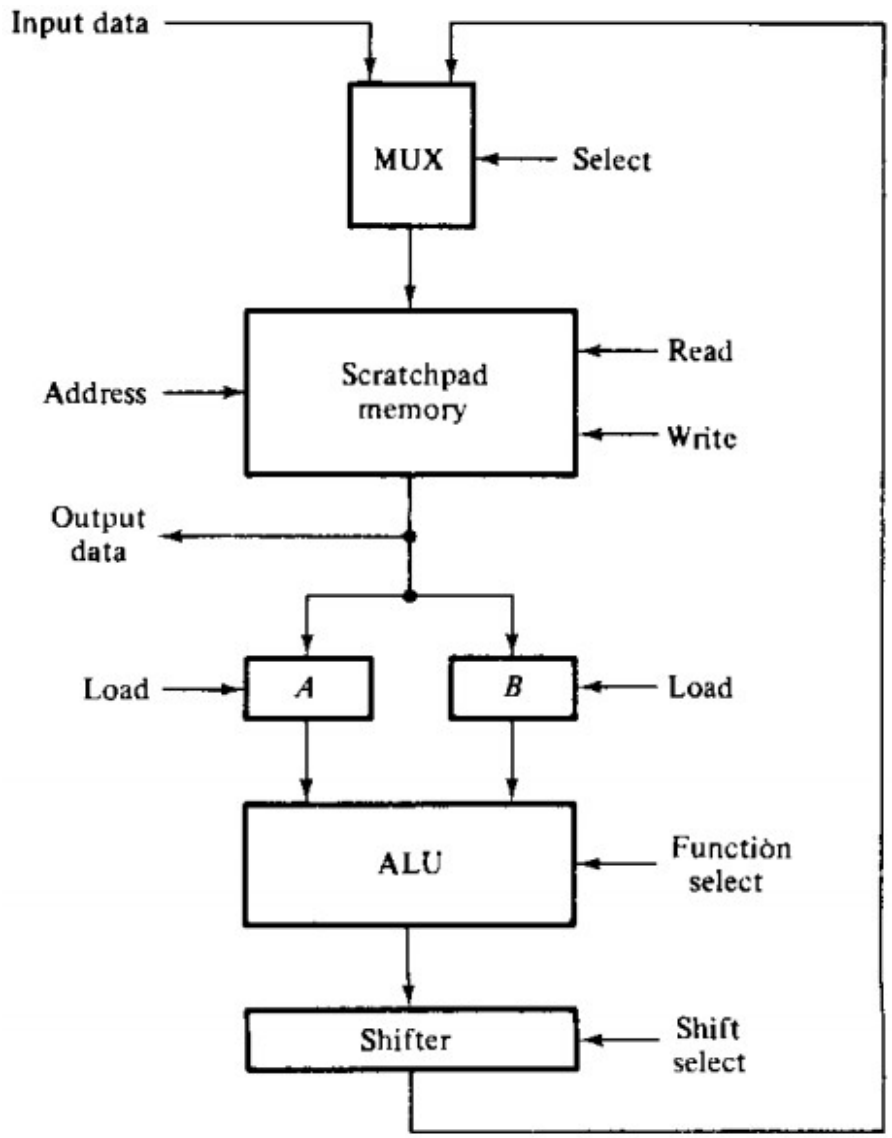
Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Processor Registers and ALU connected by Bus



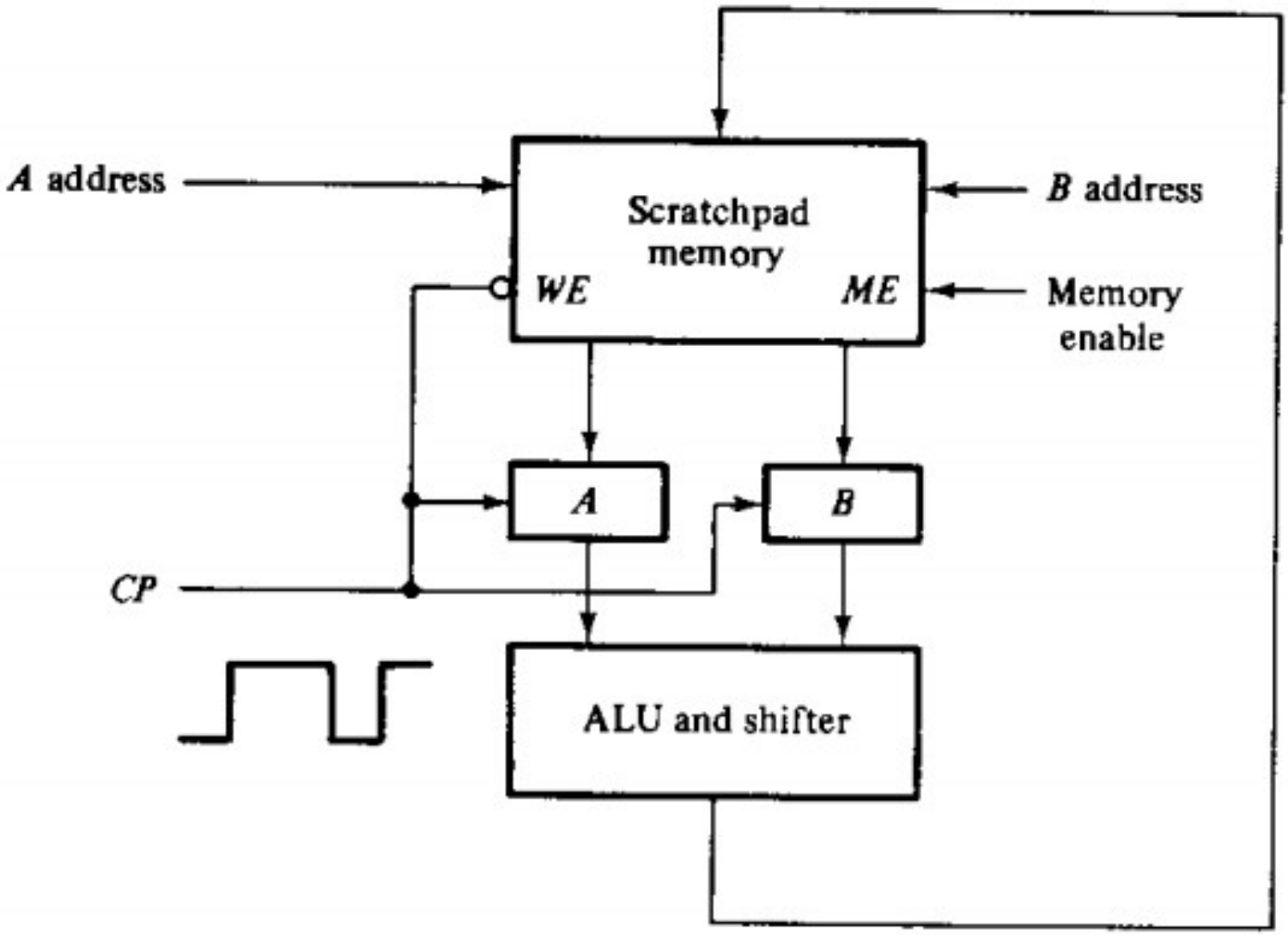
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Processor Unit with a Scratchpad Memory



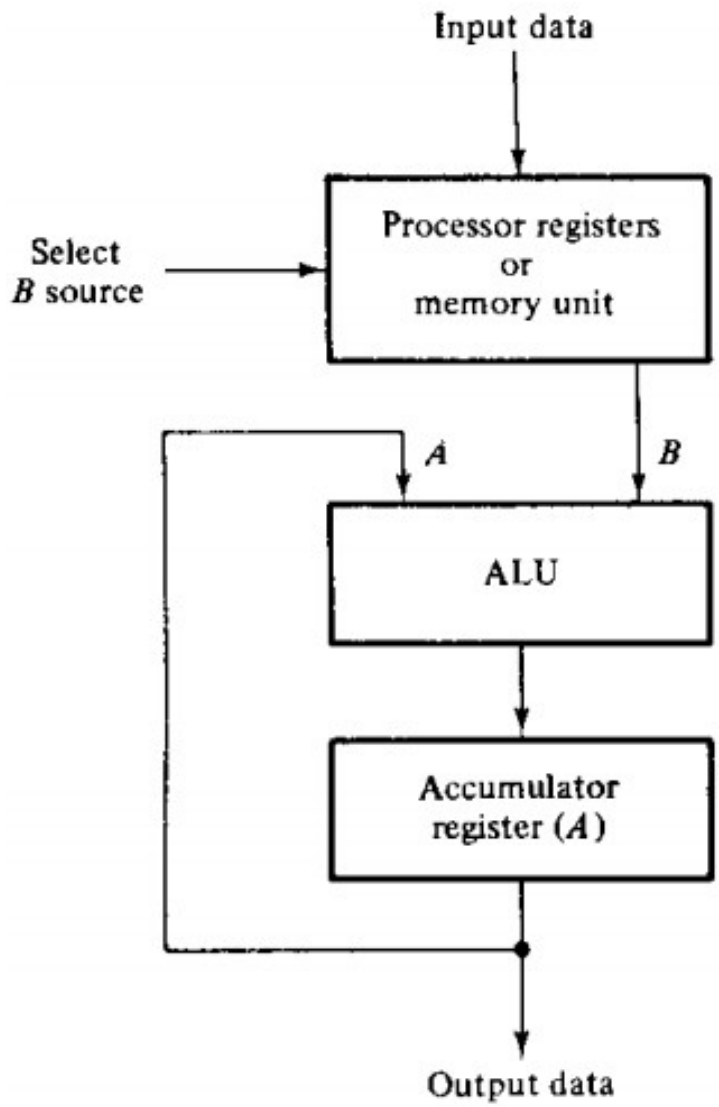
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Processor Unit with a 2-port Memory



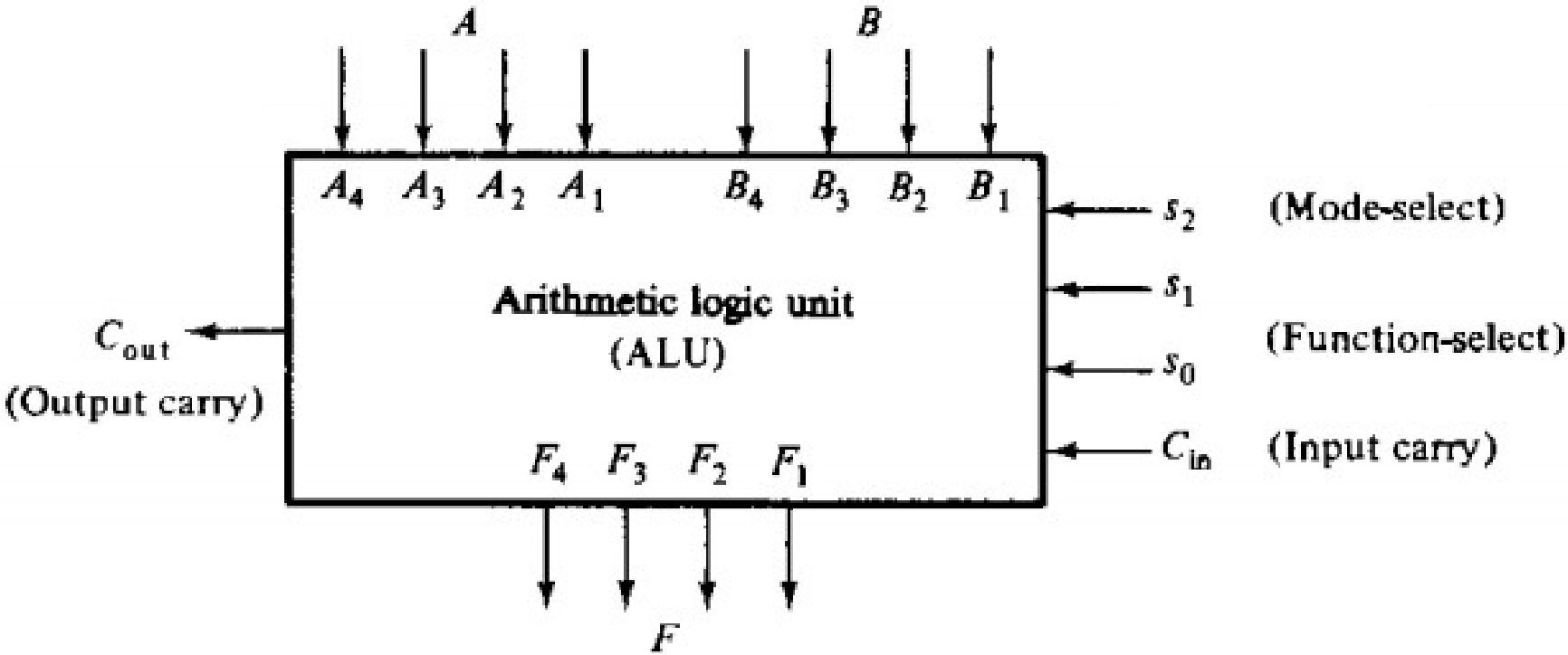
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Processor with an Accumulator Register



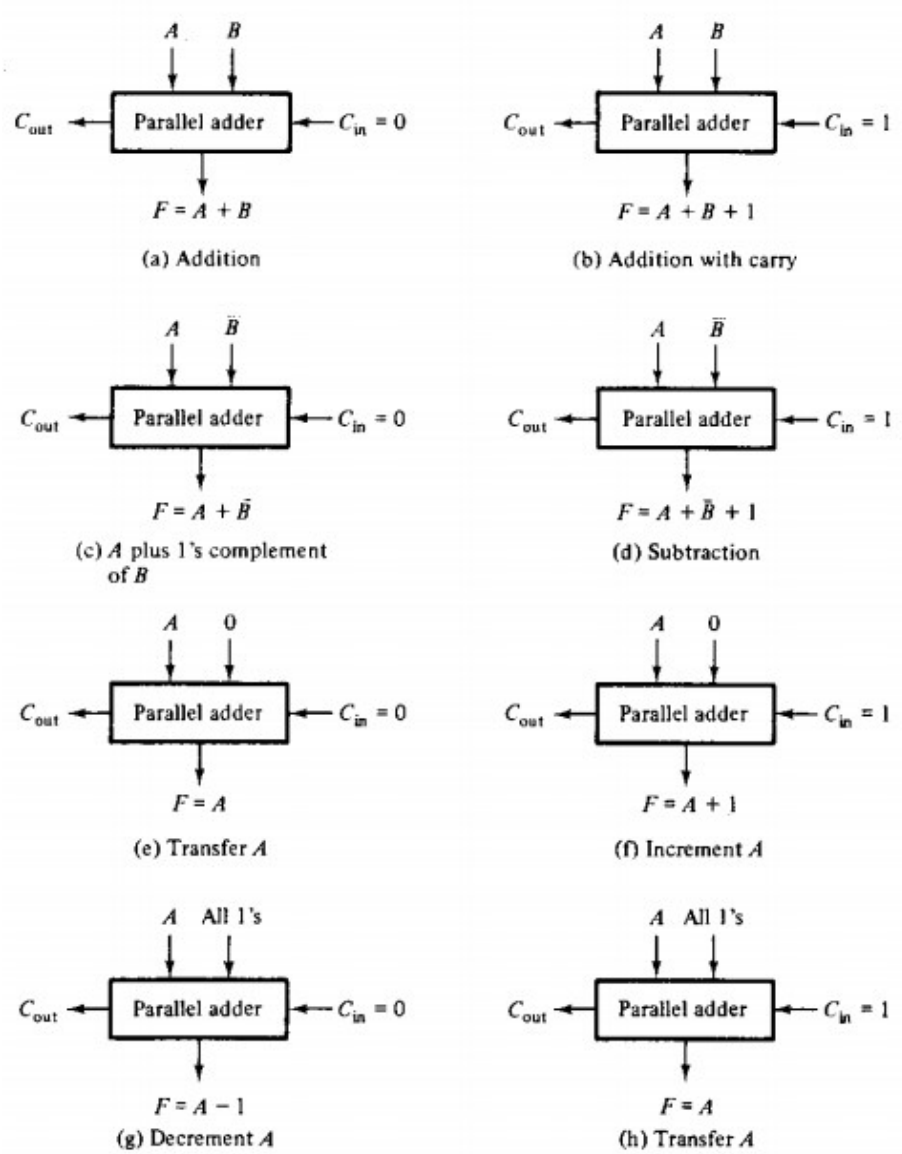
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Block Diagram of a 4-bit ALU



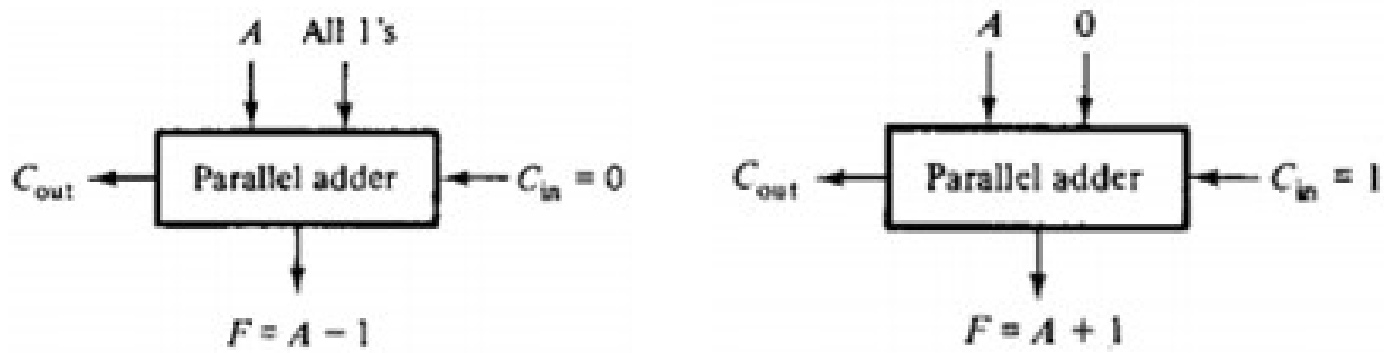
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Design of Arithmetic Circuit



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Decrement Operation



When the output carry is 1, it represents the number 2^n because 2^n in binary consists of a 1 followed by n 0's. Subtracting 1 from 2^n, we obtain $2^n - 1$, which in binary is a number of n 1's. Adding $2^n - 1$ to A, we obtain $F = A + 2^n - 1 = 2^n + A - 1$. If the output carry 2^n is removed, we obtain $F = A - 1$.

To demonstrate with a numerical example, let $n = 8$ and $A = 9$. Then:

$$
\begin{aligned}
A &= \ 0000\ \ 1001 = (9)_{10} \\
2^n &= 1\ \ 0000\ \ 0000 = (256)_{10} \\
2^n - 1 &= \ 1111\ \ 1111 = (255)_{10} \\
A + 2^n - 1 &= 1\ \ 0000\ \ 1000 = (256 + 8)_{10}
\end{aligned}
$$

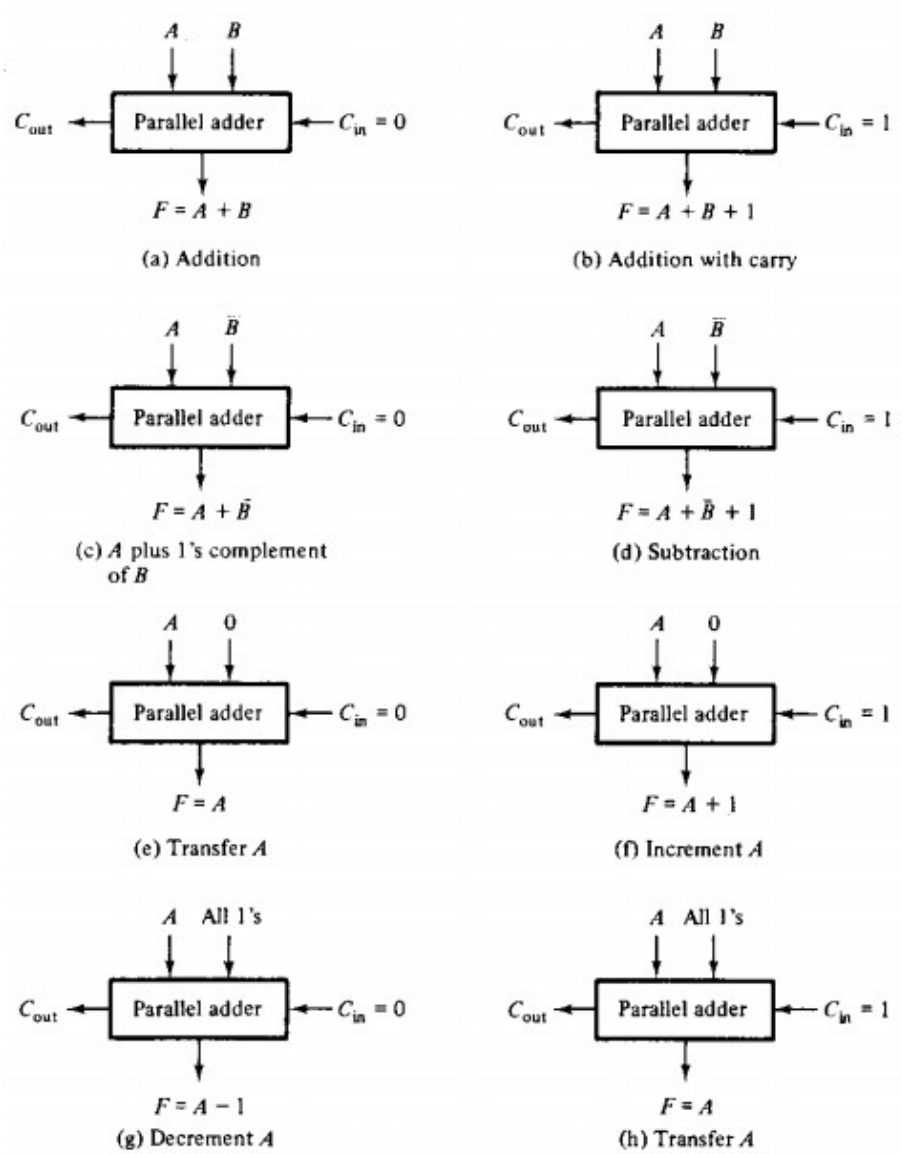
Removing the output carry $2^n = 256$, we obtain $8 = 9 - 1$. Thus, we have decremented A by 1 by adding to it a binary number with all 1's.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

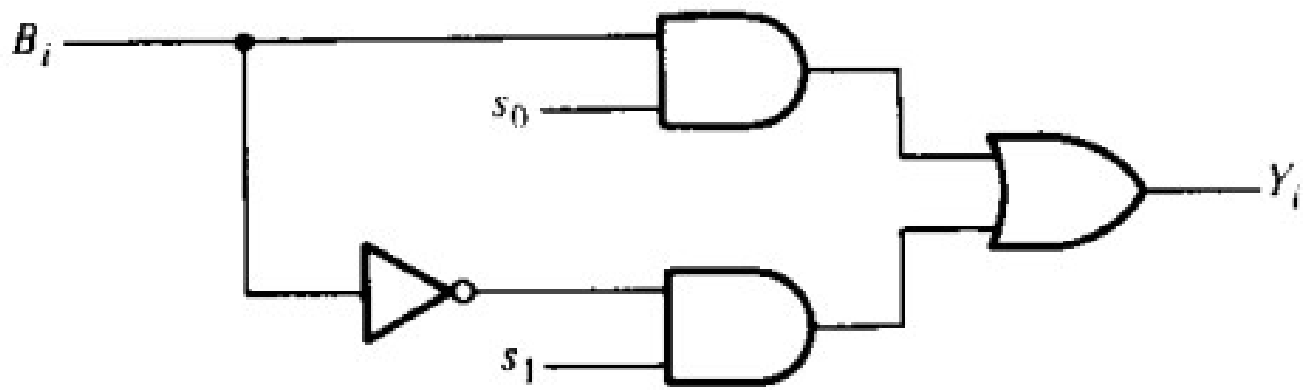
Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Design of Arithmetic Circuit



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

True/Complement, One/Zero Circuit (TCOZ)

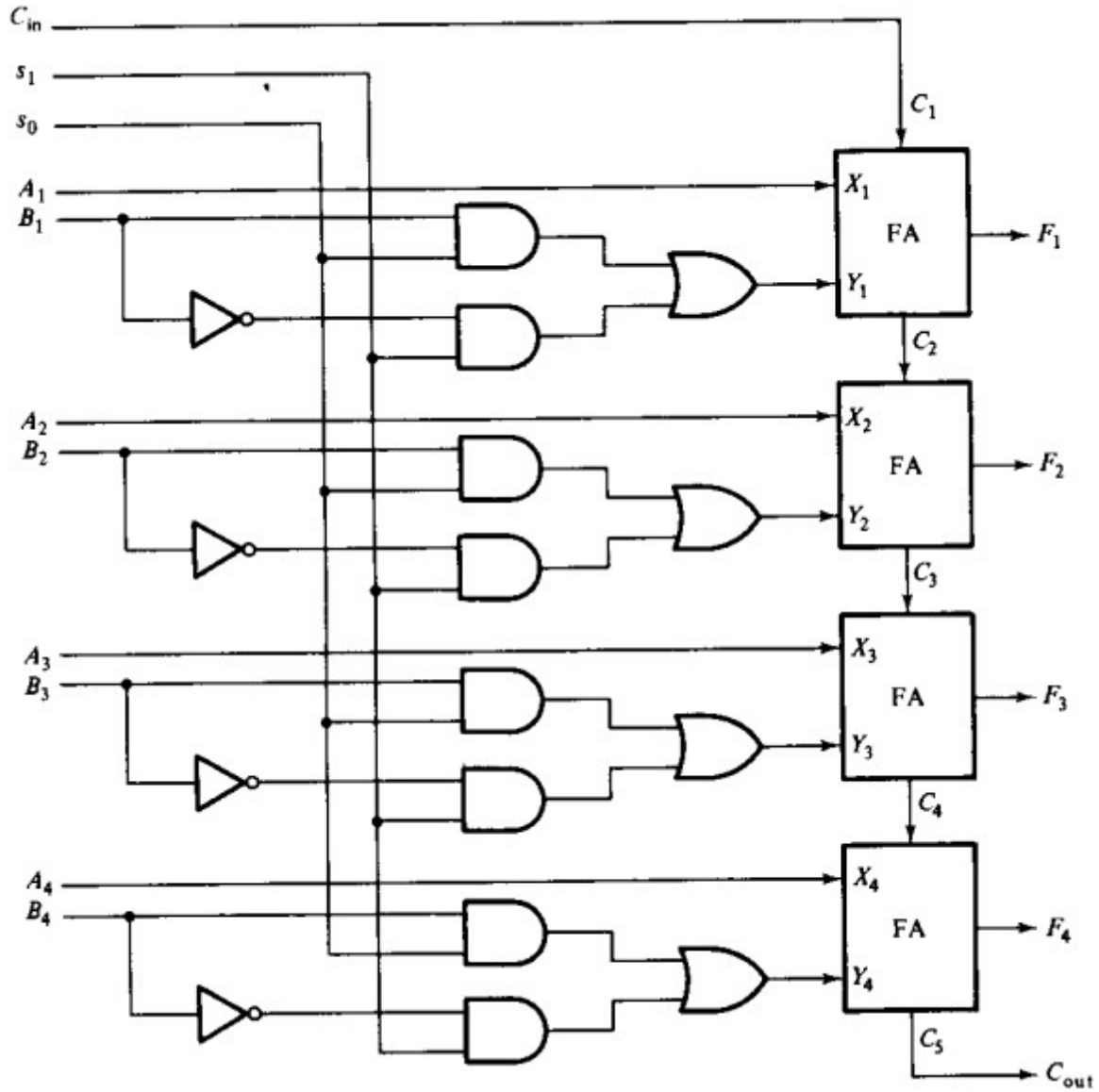


s_1	s_0	Y_i
0	0	0
0	1	B_i
1	0	B_i'
1	1	1

$$Y_i = B_i s_0 + B_i' s_1 \qquad i = 1, 2, \ldots, n$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Logic Diagram of Arithmetic Circuit



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Function Table for the Arithmetic Circuit

Function select			Y equals	Output equals	Function
s_1	s_0	C_{in}			
0	0	0	0	$F = A$	Transfer A
0	0	1	0	$F = A + 1$	Increment A
0	1	0	B	$F = A + B$	Add B to A
0	1	1	B	$F = A + B + 1$	Add B to A plus 1
1	0	0	$\overline{B}$	$F = A + \overline{B}$	Add 1's complement of B to A
1	0	1	$\overline{B}$	$F = A + \overline{B} + 1$	Add 2's complement of B to A
1	1	0	All 1's	$F = A - 1$	Decrement A
1	1	1	All 1's	$F = A$	Transfer A

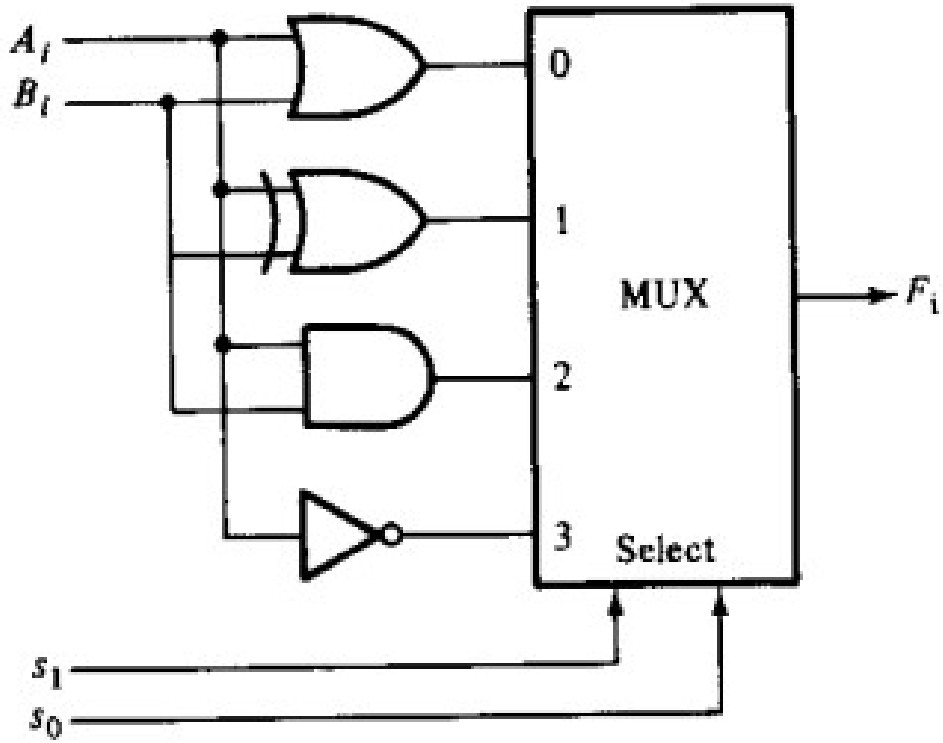
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Output Carry in the Arithmetic Circuit

Function select			Arithmetic function	$C_{out} = 1$ if	Comments
s_1	s_0	C_{in}			
0	0	0	$F = A$		C_{out} is always 0
0	0	1	$F = A + 1$	$A = 2^n - 1$	$C_{out} = 1$ and $F = 0$ if $A = 2^n - 1$
0	1	0	$F = A + B$	$(A + B) > 2^n$	Overflow occurs if $C_{out} = 1$
0	1	1	$F = A + B + 1$	$(A + B) > (2^n - 1)$	Overflow occurs if $C_{out} = 1$
1	0	0	$F = A - B - 1$	$A > B$	If $C_{out} = 0$, then $A < B$ and $F =$ 1's complement of $(B - A)$
1	0	1	$F = A - B$	$A > B$	If $C_{out} = 0$, then $A < B$ and $F =$ 2's complement of $(B - A)$
1	1	0	$F = A - 1$	$A \neq 0$	$C_{out} = 1$, except when $A = 0$
1	1	1	$F = A$		C_{out} is always 1

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

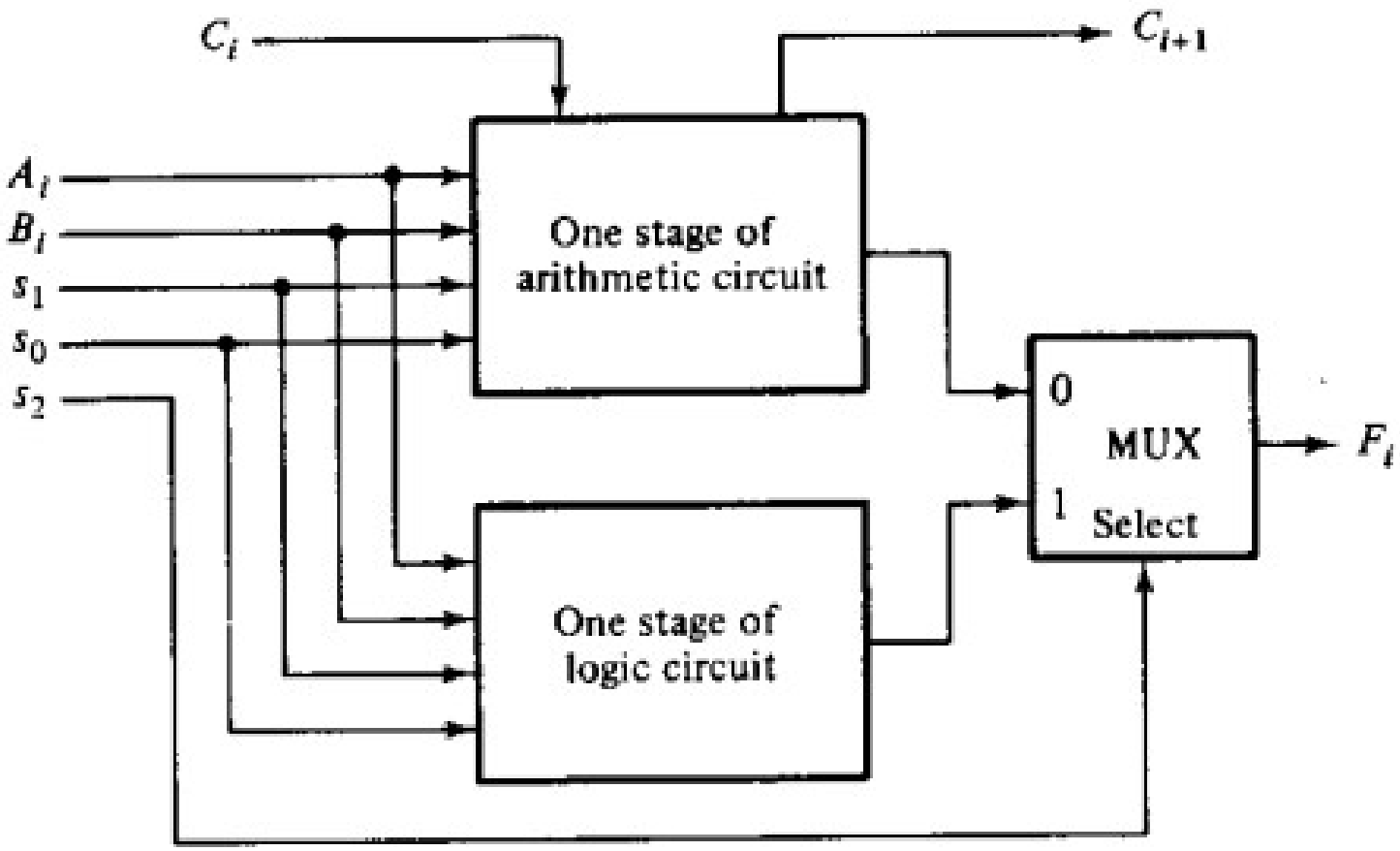
Design of Logic Circuit



s_1	s_0	Output	Operation
0	0	$F_i = A_i + B_i$	OR
0	1	$F_i = A_i \oplus B_i$	XOR
1	0	$F_i = A_i B_i$	AND
1	1	$F_i = A_i'$	NOT

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Combining Arithmetic and Logic Circuit



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Function Table for the ALU

Selection				Output	Function
s_2	s_1	s_0	C_{in}		
0	0	0	0	$F = A$	Transfer A
0	0	0	1	$F = A + 1$	Increment A
0	0	1	0	$F = A + B$	Addition
0	0	1	1	$F = A + B + 1$	Add with carry
0	1	0	0	$F = A - B - 1$	Subtract with borrow
0	1	0	1	$F = A - B$	Subtraction
0	1	1	0	$F = A - 1$	Decrement A
0	1	1	1	$F = A$	Transfer A
1	0	0	X	$F = A \vee B$	OR
1	0	1	X	$F = A \oplus B$	XOR
1	1	0	X	$F = A \wedge B$	AND
1	1	1	X	$F = \bar{A}$	Complement A

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Logic Functions using Full Adder

A more efficient ALU can be obtained if we investigate the possibility of generating logic operations in an already available arithmetic circuit. This can be done by inhibiting all input carries into the full-adder circuits of the parallel adder. Consider the Boolean function that generates the output sum in a full-adder circuit:

$$F_i = X_i \oplus Y_i \oplus C_i$$

The input carry C_i in each stage can be made to be equal to 0 when a selection variable s_2 is equal to 1. The result would be:

$$F_i = X_i \oplus Y_i$$

This expression is valid because of the property of the exclusive-OR operation $x \oplus 0 = x$. Thus, with the input carry to *each* stage equal to 0, the full-adder circuits generate the exclusive-OR operation.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Logic Operations in a Stage of Arithmetic Circuit

s_2	s_1	s_0	X_i	Y_i	C_i	$F_i = X_i \oplus Y_i$	Operation	Required operation
1	0	0	A_i	0	0	$F_i = A_i$	Transfer A	OR
1	0	1	A_i	B_i	0	$F_i = A_i \oplus B_i$	XOR	XOR
1	1	0	A_i	B_i'	0	$F_i = A_i \odot B_i$	Equivalence	AND
1	1	1	A_i	1	0	$F_i = A_i'$	NOT	NOT

s_1	s_0	Output	Operation
0	0	$F_i = A_i + B_i$	OR
0	1	$F_i = A_i \oplus B_i$	XOR
1	0	$F_i = A_i B_i$	AND
1	1	$F_i = A_i'$	NOT

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Steps in the Design of ALU

1. Design the arithmetic section independent of the logic section.
2. Determine the logic operations obtained from the arithmetic circuit in step 1, assuming that the input carries to all stages are 0.
3. Modify the arithmetic circuit to obtain the required logic operations.

To change the output to an OR operation, we must change the input to each full-adder circuit from A_i to $A_i + B_i$. This can be accomplished by ORing B_i and A_i when $s_2s_1s_0 = 100$.

The other selection variables that give an undesirable output occur when $s_2s_1s_0 = 110$. The unit as it stands generates an output $F_i = A_i \odot B_i$, but we want to generate the AND operation $F_i = A_iB_i$. Let us investigate the possibility of ORing each input A_i with some Boolean function K_i. The function so obtained is then used for X_i when $s_2s_1s_0 = 110$:

$$F_i = X_i \oplus Y_i = (A_i + K_i) \oplus B_i' = A_iB_i + K_iB_i + A_i'K_i'B_i'$$

Careful inspection of the result reveals that if the variable $K_i = B_i'$, we obtain an output: $F_i = A_iB_i + B_i'B_i + A_iB_iB_i' = A_iB_i$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Design Equations of ALU

$$X_i = A_i + s_2 s_1' s_0' B_i + s_2 s_1 s_0' B_i'$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$Z_i = s_2' C_i$$

When $s_2 = 0$, the three functions reduce to:

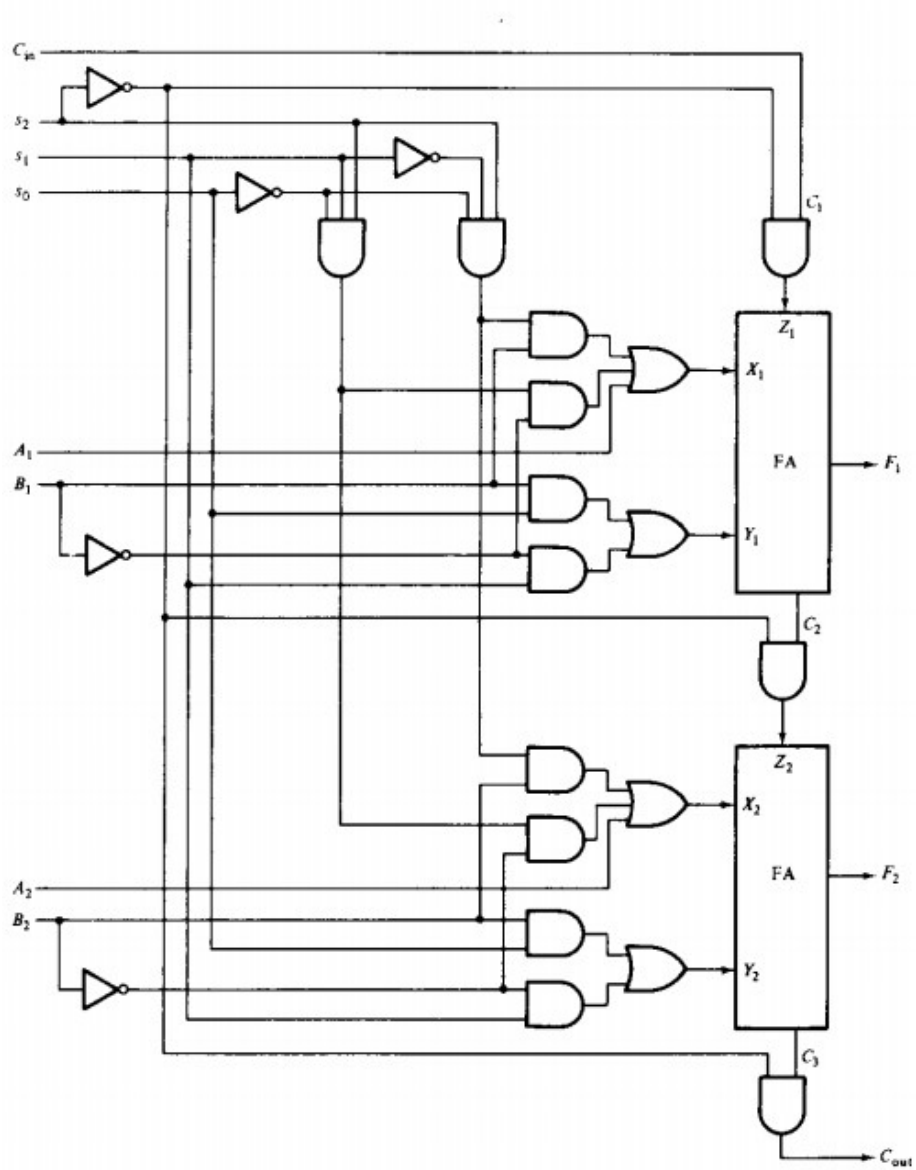
$$X_i = A_i$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$Z_i = C_i$$

For $s_2 s_1 s_0 = 101$ or 111, the functions reduce to:

$$X_i = A_i$$
$$Y_i = s_0 B_i + s_1 B_i'$$
$$C_i = 0$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Logic Diagram of ALU



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Function Table for the ALU

Selection				Output	Function
s_2	s_1	s_0	C_{in}		
0	0	0	0	$F = A$	Transfer A
0	0	0	1	$F = A + 1$	Increment A
0	0	1	0	$F = A + B$	Addition
0	0	1	1	$F = A + B + 1$	Add with carry
0	1	0	0	$F = A - B - 1$	Subtract with borrow
0	1	0	1	$F = A - B$	Subtraction
0	1	1	0	$F = A - 1$	Decrement A
0	1	1	1	$F = A$	Transfer A
1	0	0	X	$F = A \vee B$	OR
1	0	1	X	$F = A \oplus B$	XOR
1	1	0	X	$F = A \wedge B$	AND
1	1	1	X	$F = \bar{A}$	Complement A

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Status Register

1. Bit C is set if the output carry of the ALU is 1. It is cleared if the output carry is 0.
2. Bit S is set if the highest-order bit of the result in the output of the ALU (the sign bit) is 1. It is cleared if the highest-order bit is 0.
3. Bit Z is set if the output of the ALU contains all 0's, and cleared otherwise. $Z = 1$ if the result is zero, and $Z = 0$ if the result is nonzero.
4. Bit V is set if the exclusive-OR of carries C_8 and C_9 is 1, and cleared otherwise. This is the condition for overflow when the numbers are in sign-2's-complement representation. For the 8-bit ALU, V is set if the result is greater than 127 or less than -128.

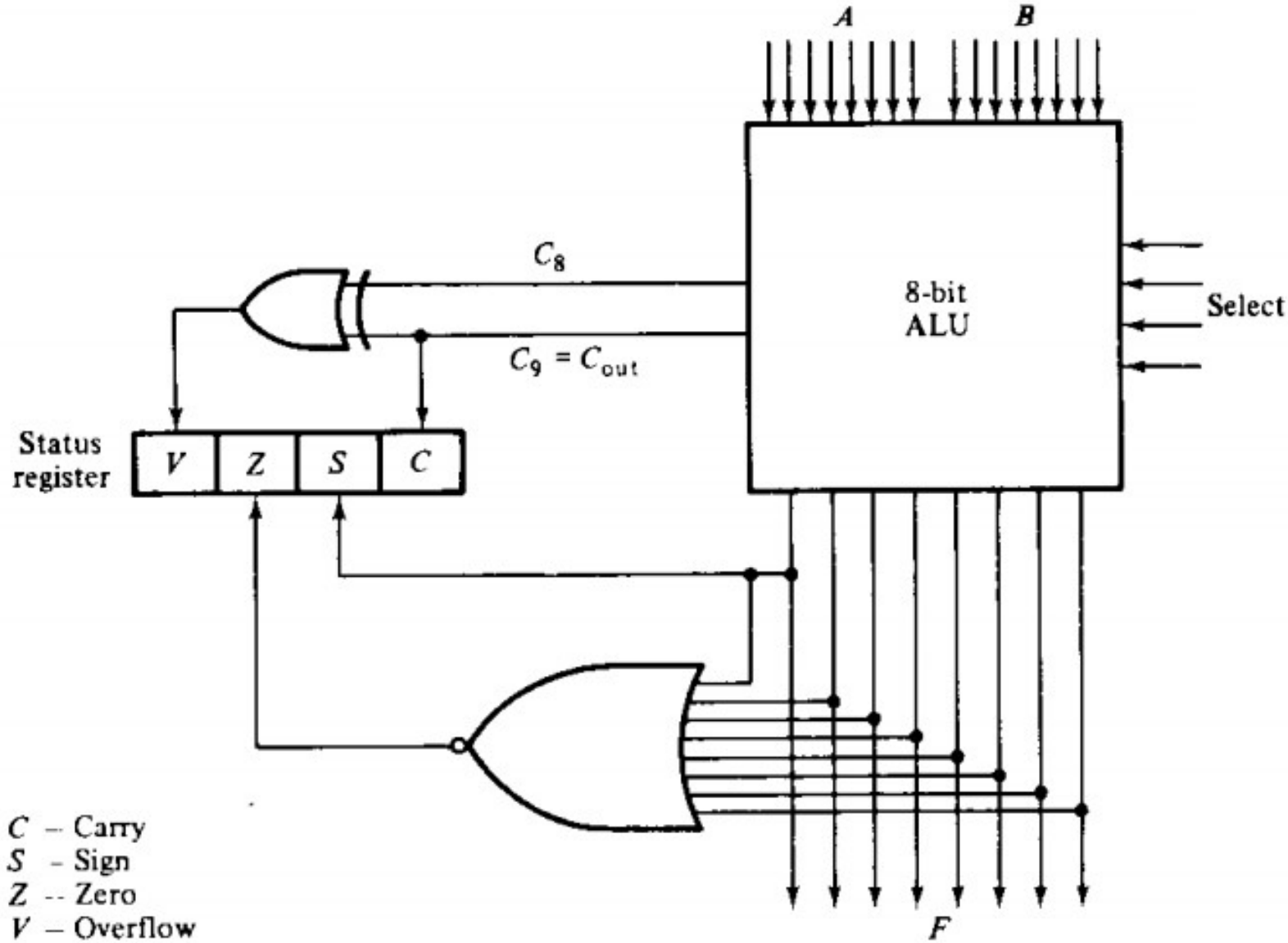
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Status Register Design

1. Bit C is set if the output carry of the ALU is 1. It is cleared if the output carry is 0.
2. Bit S is set if the highest-order bit of the result in the output of the ALU (the sign bit) is 1. It is cleared if the highest-order bit is 0.
3. Bit Z is set if the output of the ALU contains all 0's, and cleared otherwise. $Z = 1$ if the result is zero, and $Z = 0$ if the result is nonzero.
4. Bit V is set if the exclusive-OR of carries C_8 and C_9 is 1, and cleared otherwise. This is the condition for overflow when the numbers are in sign-2's-complement representation. For the 8-bit ALU, V is set if the result is greater than 127 or less than -128.

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Status Register



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Status Bits after the subtraction of Unsigned Numbers (A-B)

Relation	Condition of status bits	Boolean function
$A > B$	$C = 1$ and $Z = 0$	CZ'
$A \geq B$	$C = 1$	C
$A < B$	$C = 0$	C'
$A \leq B$	$C = 0$ or $Z = 1$	$C' + Z$
$A = B$	$Z = 1$	Z
$A \neq B$	$Z = 0$	Z'

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Status Bits after the subtraction of Signed-2's Complement Numbers (A-B)

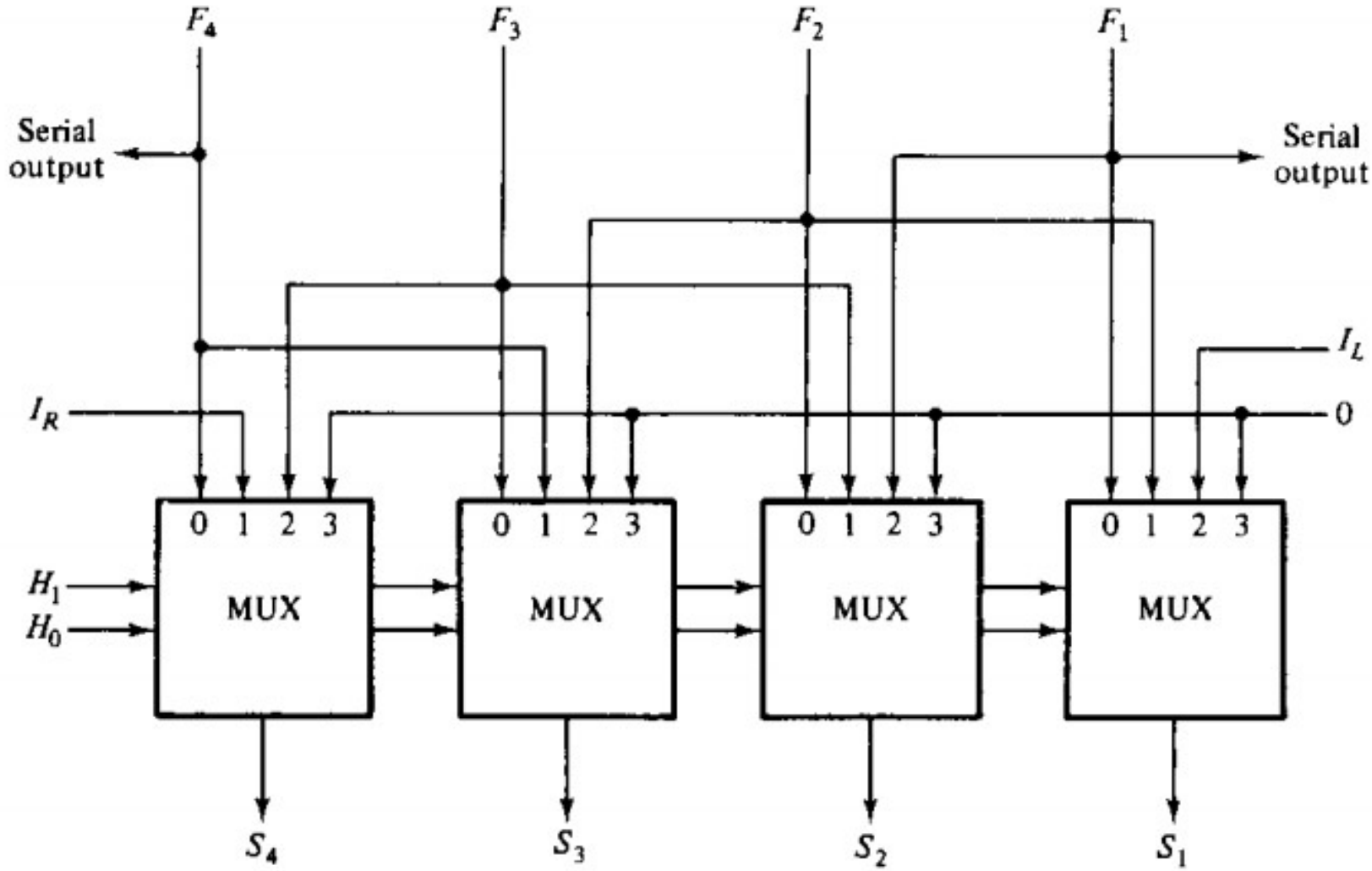
Relation	Condition of status bits	Boolean function
$A > B$	$Z = 0$ and ($S = 0$, $V = 0$ or $S = 1$, $V = 1$)	$Z'(S \odot V)$
$A \geqslant B$	$S = 0$, $V = 0$ or $S = 1$, $V = 1$	$S \odot V$
$A < B$	$S = 1$, $V = 0$ or $S = 0$, $V = 1$	$S \oplus V$
$A \leqslant B$	$S = 1$, $V = 0$ or $S = 0$, $V = 1$ or $Z = 1$	$(S \oplus V) + Z$
$A = B$	$Z = 1$	Z
$A \neq B$	$Z = 0$	Z'

Reference: M. Morris Mano, "Digital Logic and Computer Design", PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

4-bit Combinational Logic Shifter



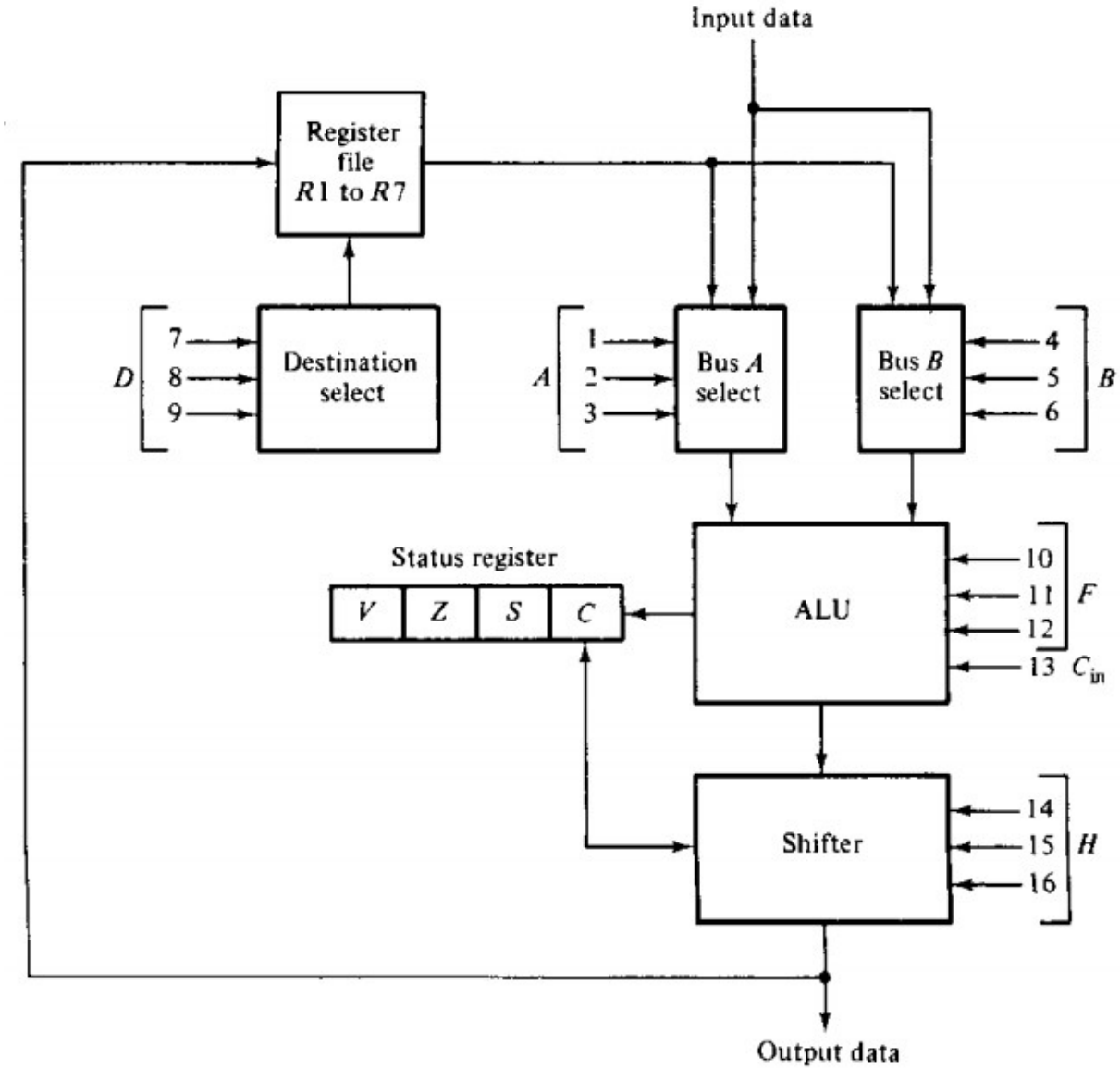
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Function Table for the Shifter

H_1	H_0	Operation	Function
0	0	$S \leftarrow F$	Transfer F to S (no shift)
0	1	$S \leftarrow \text{shr } F$	Shift-right F into S
1	0	$S \leftarrow \text{shl } F$	Shift-left F into S
1	1	$S \leftarrow 0$	Transfer 0's into S

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Processor Unit



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Control Variables

Control word

1 2 3	4 5 6	7 8 9	10 11 12	13	14 15 16
A	B	D	F	C_{in}	H

Binary code	Function of selection variables					
	A	B	D	F with $C_{in} = 0$	F with $C_{in} = 1$	H
0 0 0	Input data	Input data	None	$A, C \leftarrow 0$	$A + 1$	No shift
0 0 1	$R1$	$R1$	$R1$	$A + B$	$A + B + 1$	Shift-right, $I_R = 0$
0 1 0	$R2$	$R2$	$R2$	$A - B - 1$	$A - B$	Shift-left, $I_L = 0$
0 1 1	$R3$	$R3$	$R3$	$A - 1$	$A, C \leftarrow 1$	0's to output bus
1 0 0	$R4$	$R4$	$R4$	$A \vee B$	—	—
1 0 1	$R5$	$R5$	$R5$	$A \oplus B$	—	Circulate-right with C
1 1 0	$R6$	$R6$	$R6$	$A \wedge B$	—	Circulate-left with C
1 1 1	$R7$	$R7$	$R7$	$\bar{A}$	—	—

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Examples of Micro-operations for the Processor

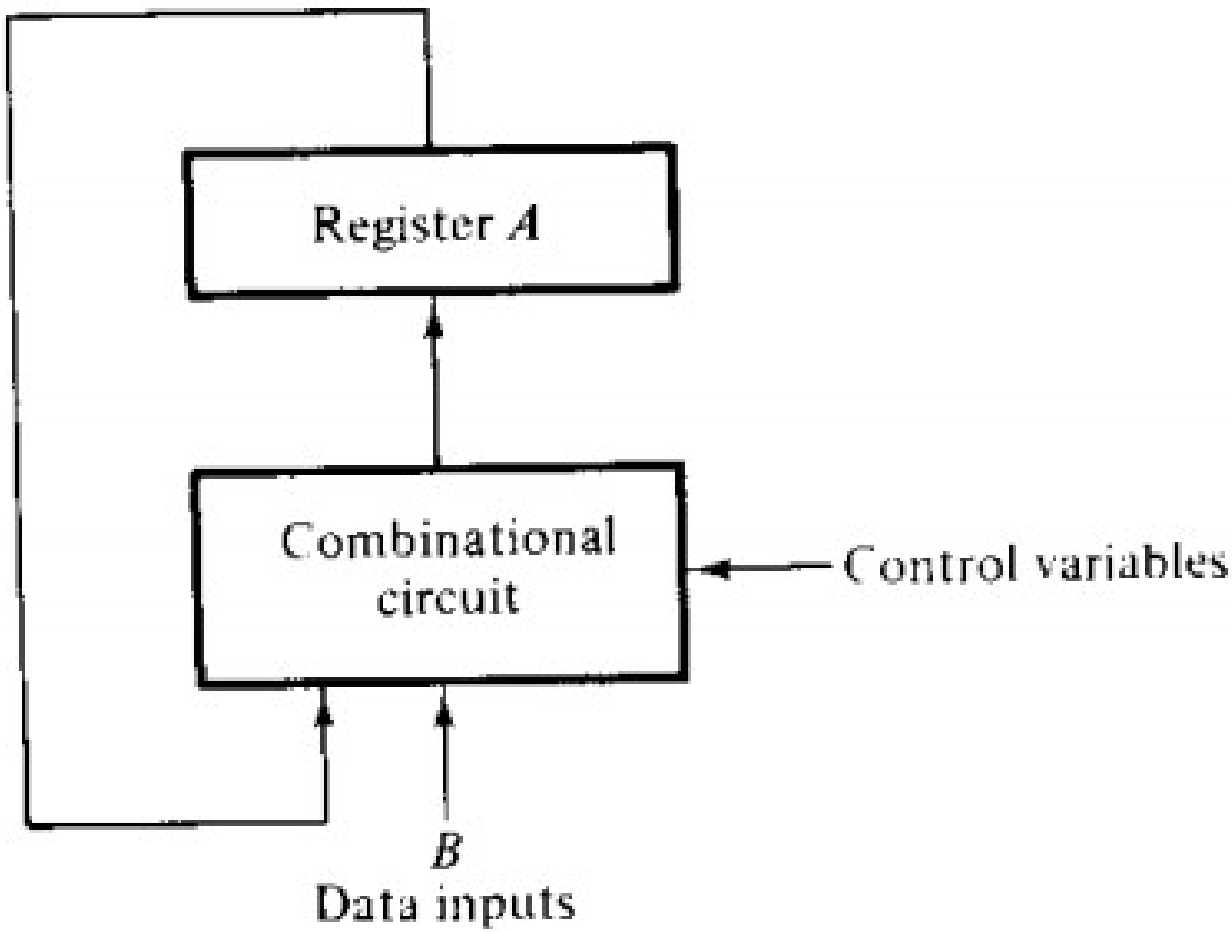
Microoperation	Control word						Function
	A	B	D	F	C_{in}	H	
$R1 \leftarrow R1 - R2$	001	010	001	010	1	000	Subtract $R2$ from $R1$
$R3 - R4$	011	100	000	010	1	000	Compare $R3$ and $R4$
$R5 \leftarrow R4$	100	000	101	000	0	000	Transfer $R4$ to $R5$
$R6 \leftarrow$ Input	000	000	110	000	0	000	Input data to $R6$
Output $\leftarrow R7$	111	000	000	000	0	000	Output data from $R7$
$R1 \leftarrow R1, C \leftarrow 0$	001	000	001	000	0	000	Clear carry bit C
$R3 \leftarrow$ shl $R3$	011	011	011	100	0	010	Shift-left $R3$ with $I_L = 0$
$R1 \leftarrow$ crc $R1$	001	001	001	100	0	101	Circulate-right $R1$ with carry
$R2 \leftarrow 0$	000	000	010	000	0	011	Clear $R2$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

COC2070 – Digital Logic and System Design

Prof. M. M. Sufyan Beg
Department of Computer Engineering
Z. H. College of Engineering & Technology
Aligarh Muslim University, India

Processor Unit with an Accumulator



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Micro-operations for an Accumulator

Control variable	Microoperation	Name
p_1	$A \leftarrow A + B$	Add
p_2	$A \leftarrow 0$	Clear
p_3	$A \leftarrow \overline{A}$	Complement
p_4	$A \leftarrow A \wedge B$	AND
p_5	$A \leftarrow A \vee B$	OR
p_6	$A \leftarrow A \oplus B$	Exclusive-OR
p_7	$A \leftarrow \text{shr } A$	Shift-right
p_8	$A \leftarrow \text{shl } A$	Shift-left
p_9	$A \leftarrow A + 1$	Increment
	If $(A = 0)$ then $(Z = 1)$	Check for zero

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Add B to A (p_1)

Present state	Inputs		Next state	Flip-flop inputs		Output
A_i	B_i	C_i	A_i	JA_i	KA_i	C_{i+1}
0	0	0	0	0	X	0
0	0	1	1	1	X	0
0	1	0	1	1	X	0
0	1	1	0	0	X	1
1	0	0	1	X	0	0
1	0	1	0	X	1	1
1	1	0	0	X	1	1
1	1	1	1	X	0	1

$$JA_i = B_i C_i' p_1 + B_i' C_i p_1$$
$$KA_i = B_i C_i' p_1 + B_i' C_i p_1$$
$$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$$

B_i

	1		1
X	X	X	X

A_i

C_i

$JA_i = B_i C_i' + B_i' C_i$

X	X	X	X
	1		1

$KA_i = B_i C_i' + B_i' C_i$

		1	
	1	1	1

$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Clear and Complement

Clear (p_2)

$JA_i = 0$

$KA_i = p_2$

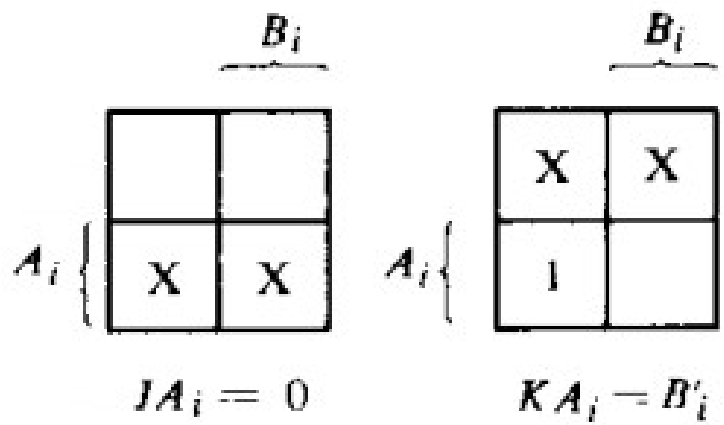
Complement (p_3)

$JA_i = p_3$

$KA_i = p_3$

AND (p_4)

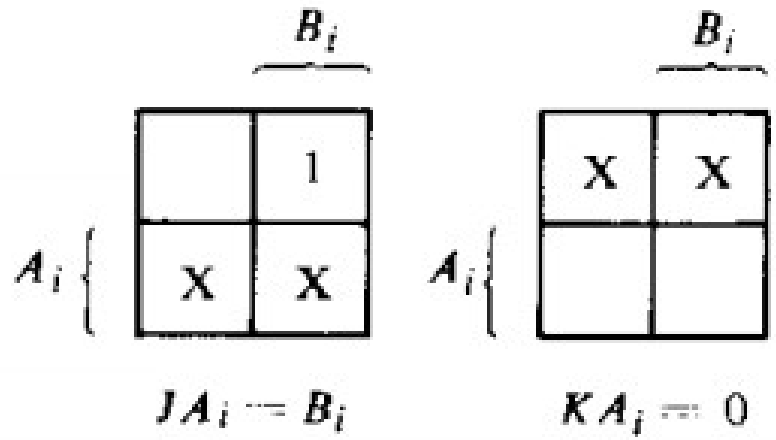
Present state	Input	Next state	Flip-flop inputs	
A_i	B_i	A_i	JA_i	KA_i
0	0	0	0	X
0	1	0	0	X
1	0	0	X	1
1	1	1	X	0



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

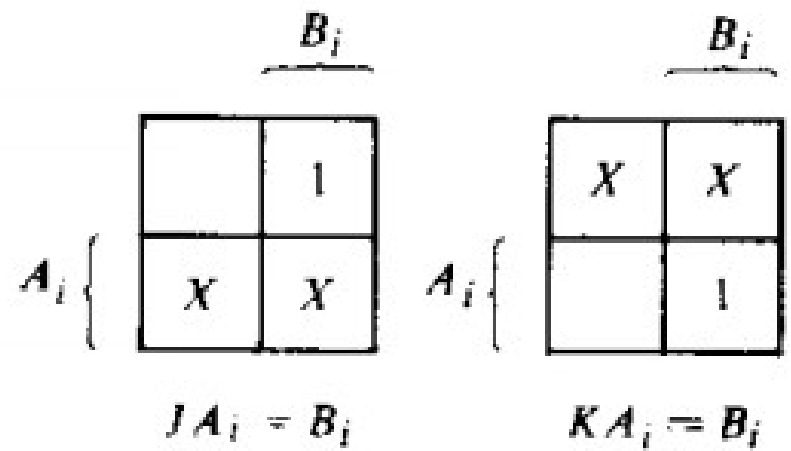
OR (p_5)

Present state	Input	Next state	Flip-flop inputs	
A_i	B_i	A_i	JA_i	KA_i
0	0	0	0	X
0	1	1	1	X
1	0	1	X	0
1	1	1	X	0



Exclusive-OR (p_6)

Present state	Input	Next state	Flip-flop inputs	
A_i	B_i	A_i	JA_i	KA_i
0	0	0	0	X
0	1	1	1	X
1	0	1	X	0
1	1	0	X	1



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Shift-right (p_7)

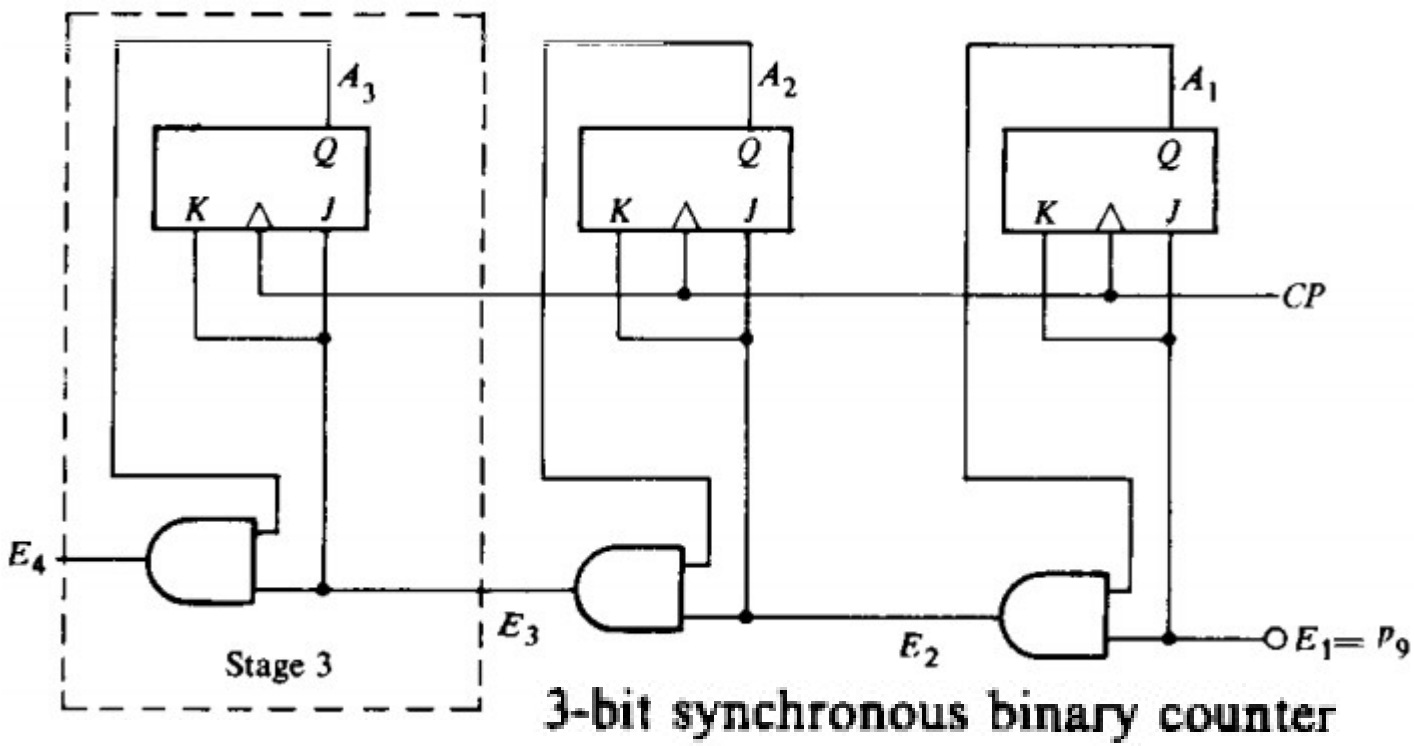
$$JA_i = A_{i+1}p_7$$
$$KA_i = A'_{i+1}p_7$$

Shift-left (p_8)

$$JA_i = A_{i-1}p_8$$
$$KA_i = A'_{i-1}p_8$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Increment (p_9)



$$JA_i = E_i$$

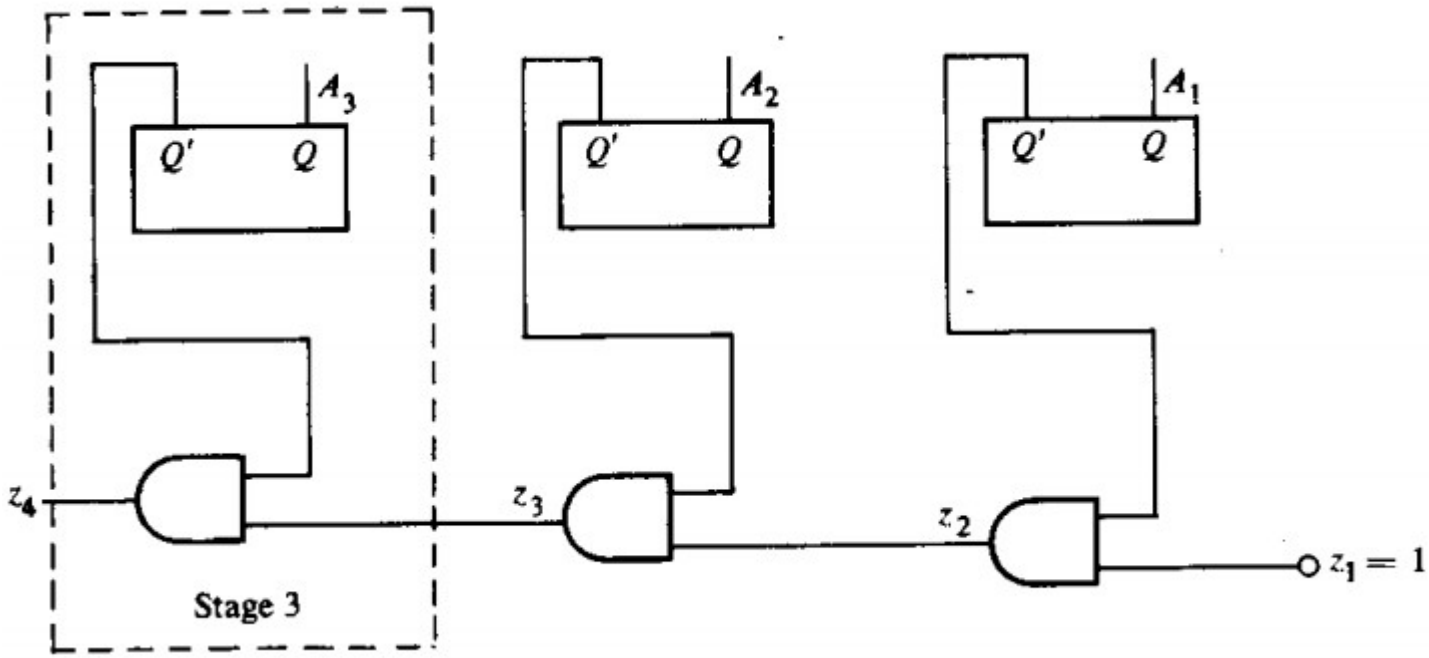
$$KA_i = E_i$$

$$E_{i+1} = E_i A_i \qquad i = 1, 2, \ldots, n$$

$$E_1 = p_9$$

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Check for Zero (Z)



Chain of AND gates for checking the zero content of a register

$$z_{i+1} = z_i A_i' \qquad i = 1, 2, \ldots, n$$

$$z_1 = 1$$

$$z_{n+1} = Z$$

Variable Z becomes 1 if the output signal from the last stage, z_{n+1}, is 1

Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

One Stage of Accumulator

Combining all the input functions for the J and the K inputs of flip-flop A_i produces a composite set of input Boolean functions for a typical stage:

$$JA_i = B_iC_i'p_1 + B_i'C_ip_1 + p_3 + B_ip_5 + B_ip_6 + A_{i+1}p_7 + A_{i-1}p_8 + E_i$$

$$KA_i = B_iC_i'p_1 + B_i'C_ip_1 + p_2 + p_3 + B_i'p_4 + B_ip_6 + A_{i+1}'p_7 + A_{i-1}'p_8 + E_i$$

Each stage in the accumulator must also generate the carries for the next stage:

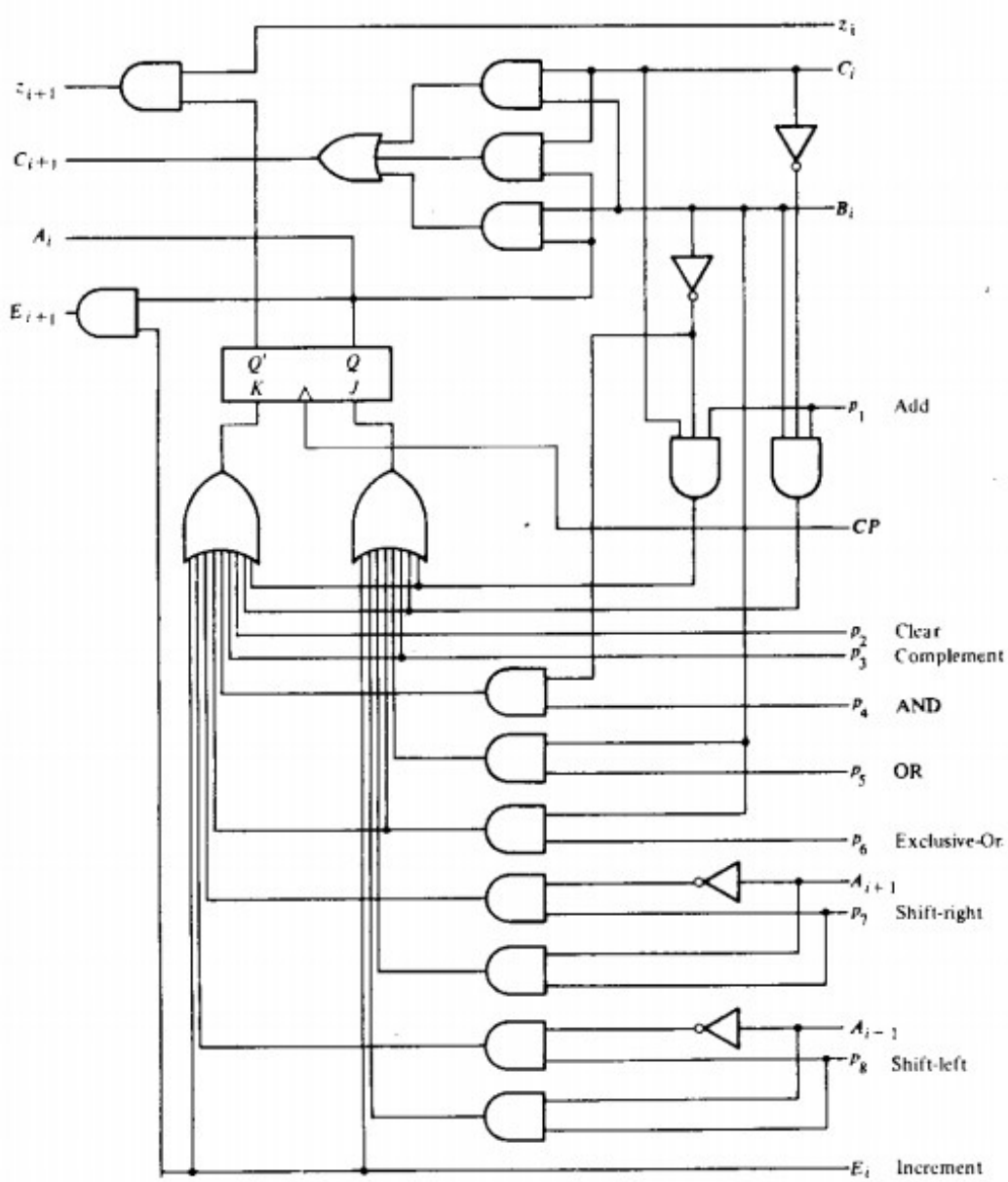
$$C_{i+1} = A_iB_i + A_iC_i + B_iC_i$$

$$E_{i+1} = E_iA_i$$

$$z_{i+1} = z_iA_i'$$

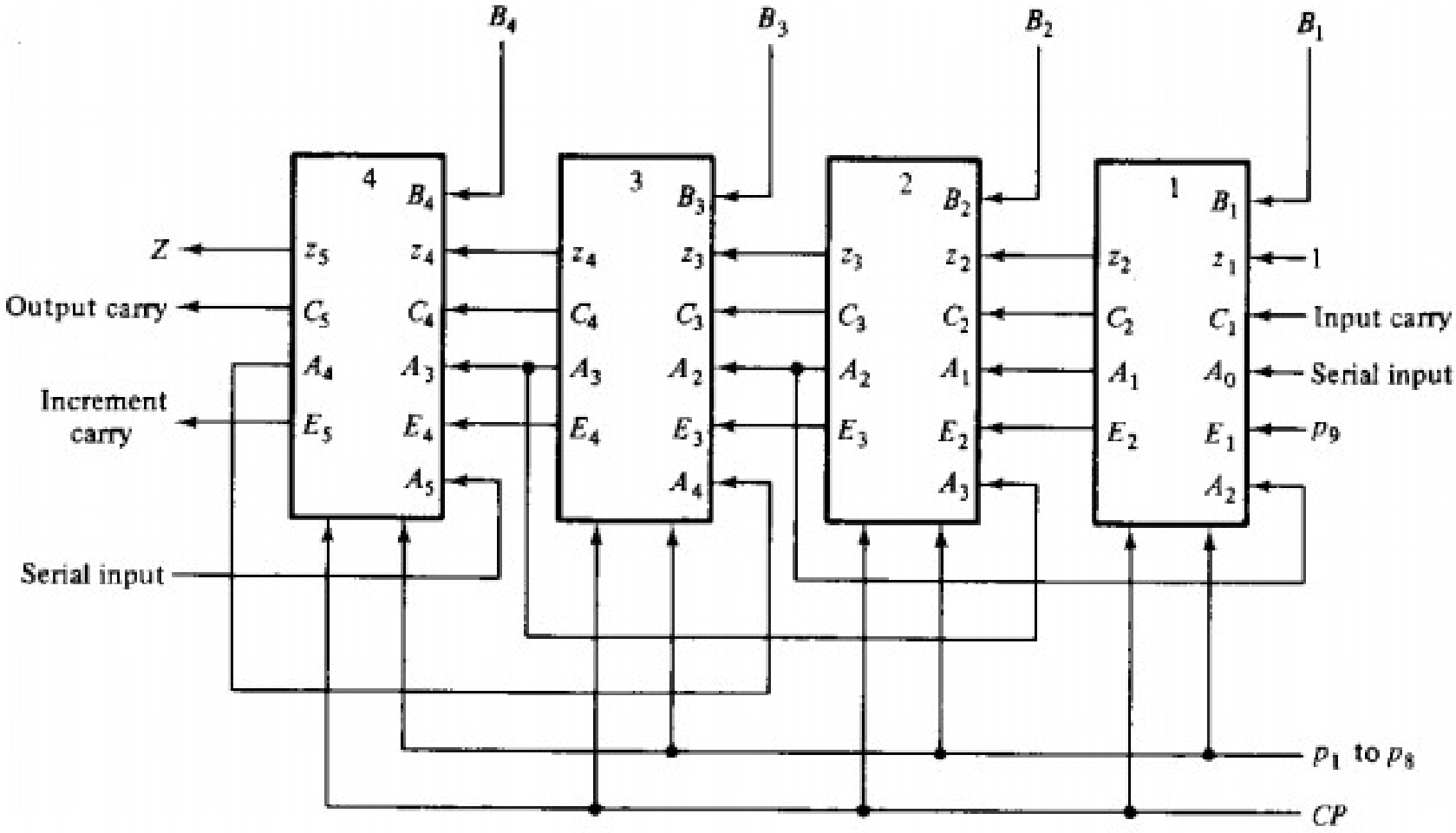
Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

One Stage of Accumulator



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

4-bit Accumulator



Reference: M. Morris Mano, “Digital Logic and Computer Design”, PHI

Unit-IV Completed

UNIT IV.

REGISTER TRANSFER LOGIC

Introduction to Register Transfer Logic, macro-operations and micro-operations, introduction to control unit, design and hardware specification of a simple computer, instruction fetch and execute cycle, Introduction to ALU design.

Course Outcome

4. Design and implement simple digital system comprising control unit and data path.